# 异构数据库迁移从入门到精通

> **版本与核实信息**
>
> - 核实日期：2026-10-25。本教程中的版本号、命令与配置项均于该日期通过官方文档与官方发布说明核实，无法核实的细节一律省略或标注「以官方文档为准」。
> - 版本基线：Addax 6.2.0、Apache SeaTunnel 2.3.13（3.0.0 已于 2026-09-29 发布）、Apache Flink CDC 3.7.0（配套 Flink 1.20.x 与 2.2.x）、Debezium 3.7.0.Final、pgloader v3.6.x 稳定版（v4 为预发布）、Canal 1.1.8、mydumper/myloader 1.0.x、SymmetricDS 社区版 3.16.x。
> - 数据库基线：MySQL 5.7 与 8.0.x、8.4.x，PostgreSQL 12 至 18，SQL Server 2016 SP1 及以上（Standard 或 Enterprise 版本）。
> - 目标读者：具备基础 SQL 与 Linux 命令行经验的后端工程师、DBA 与数据工程师。
> - 案例约定：下文统一使用 `prod` 表示生产或旧环境数据库，`test` / `new` 表示测试或新环境数据库；示例中出现的密码均为演示占位符，生产环境应使用密钥管理服务下发凭据。
> - 图示说明：本教程使用 Mermaid 代码块绘制架构图与流程图，在 GitHub、GitLab、Typora、VS Code 等支持 Mermaid 的阅读器中可直接渲染。

## 前言

数据库迁移是工程实践中风险最高的操作类型之一：数据是不可再生资产，任何一步误操作都可能造成不可逆损失。本教程围绕四类真实需求组织内容——异构数据库间的迁移（如 MySQL 迁往 PostgreSQL、SQL Server 迁往 MySQL）、只迁移部分表、按条件筛选迁移部分数据、以及在业务不停服的前提下实时迁移。教程以开源免费方案为主线，将业界生产级方案归纳为四类：原生工具与脚本、批式同步引擎、实时 CDC 同步、双向持续复制。

阅读方法建议如下：第一篇建立整体认知并完成选型；第二篇按方案深入学习，每个方案内部自成「入门、进阶、精通」闭环，读者也可以按需直接跳转到候选方案对应的章节；第三篇补充跨方案的专题能力；第四篇面向生产落地。全文示例命令均针对 Linux 环境编写，Windows 环境下 SQL Server 相关命令已单独标注。

---

# 第一篇 认知篇

## 第 1 章 数据迁移基础与核心概念

本章解决三个问题：数据迁移到底有哪些类型、一次迁移在技术上由哪些环节组成、描述迁移方案时频繁出现的术语分别是什么含义。完成本章学习后，读者应能准确判断自己面对的迁移属于哪一类，并理解后续章节中方案对比的讨论基础。

### 1.1 迁移场景分类

按照源与目标的数据库引擎是否相同，迁移分为两类。

**同构迁移**（homogeneous migration）：源库与目标库使用同一引擎，例如 MySQL 5.7 迁往 MySQL 8.4，SQL Server 2016 迁往 SQL Server 2022。此类迁移通常可以借助引擎自带的备份恢复、逻辑导出导入或原生复制能力完成，类型映射几乎不存在障碍。

**异构迁移**（heterogeneous migration）：源库与目标库使用不同引擎，例如 MySQL 迁往 PostgreSQL、SQL Server 迁往 MySQL、PostgreSQL 迁往 Doris。此类迁移除数据本身外，还必须处理三类差异：数据类型系统差异（如 MySQL 的 `unsigned` 整型在 PostgreSQL 中没有直接对应物）、SQL 方言与函数差异、以及对象语义差异（如 MySQL 的自增列对应 PostgreSQL 的序列，存储过程与触发器通常无法自动转换）。

按照迁移的时空约束，又分为两类。

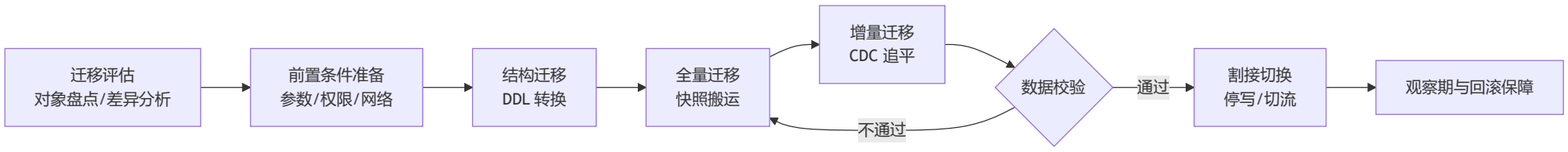
**一次性搬迁**：迁移完成后源库退役或转为只读，常见于旧环境向新环境迁移、上云迁移。此类迁移允许出现停机窗口，也可以通过增量追平把窗口压缩到分钟级。

**持续同步**：迁移完成后两个库长期共存并保持数据一致，常见于生产环境向测试环境定期分发数据、异构库之间保持镜像（例如关系库向分析库、搜索库持续投递变更）。

生产环境向测试环境迁移是两类约束的组合：通常是异构或同构的一次性子集搬迁，并叠加脱敏要求；旧环境向新环境迁移则往往要求接近零停机，属于增量追平加割接的一次性搬迁。

### 1.2 一次迁移的技术环节

无论采用哪种工具，一次生产级迁移都由以下环节构成，区别只在于各环节由工具自动完成还是由人工完成。

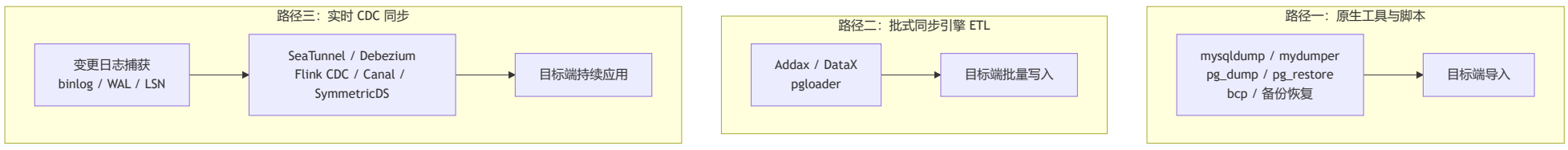



各环节说明如下。

1. **迁移评估**：盘点需要迁移的库、表、数据量与特殊对象（大对象、无主键表、序列、存储过程），评估源库与目标库的类型兼容性，确定停机预算。
2. **前置条件准备**：开启源端的变更日志能力（MySQL 的 binlog、PostgreSQL 的逻辑 WAL、SQL Server 的 CDC），创建具备相应权限的账号，打通网络连通性。
3. **结构迁移**：在目标库创建库、表、索引等对象。异构场景下这一步通常需要人工介入或借助工具的类型映射。
4. **全量迁移**（full load）：把存量数据一次性搬到目标库。
5. **增量迁移**（incremental replication）：把全量开始之后源端发生的变更持续同步到目标端，使两端数据逐渐追平。
6. **数据校验**：对行数、聚合值、抽样明细进行比对，确认两端一致。
7. **割接切换**（cutover）：把业务流量从源库切换到目标库，是整个迁移中唯一不可逆的窗口，必须预先设计回滚路径。
8. **观察与回滚保障**：割接后保留反向同步或源库只读实例一段时间，发现问题可以回退。

## 1.3 三条技术路径

实现上述环节的工具方案可以归纳为三条路径，本教程第二篇的四个方案分别落在这些路径上。




- **路径一**利用数据库自带的导出导入工具，零额外依赖，适合可停机的中小规模迁移，行级筛选能力强（本质上就是执行带条件的查询），但异构场景下类型转换需要人工处理。
- **路径二**利用独立的数据同步引擎，通过 JDBC 等协议并行抽取和写入，天然支持跨引擎异构、列裁剪与行级条件筛选，吞吐可控，但不捕获变更日志，无法自动追平全量期间的新增写入，适合做全量搬运与周期性批量同步。
- **路径三**通过解析数据库的事务日志实现变更捕获（Change Data Capture，CDC），对源库侵入小、延迟可达秒级，是实时迁移与不停服迁移的基础；生产级落地通常需要「全量快照 + 增量流式」一体化的工具链。

三条路径不是互斥的。生产级不停服迁移的常见组合是：用路径一或路径二完成全量搬运（速度快、吞吐高），用路径三从全量的快照位点开始追平增量，最后割接。第 2 章与第 10 章将分别给出选型方法与组合实战。

## 1.4 核心术语

| 术语 | 英文 | 含义 |
| --- | --- | --- |
| 全量迁移 | full load / initial snapshot | 将存量数据一次性复制到目标端 |
| 增量迁移 | incremental replication | 将源端后续发生的变更持续应用到目标端 |
| 变更数据捕获 | Change Data Capture（CDC） | 通过解析事务日志或触发器捕获行级增删改事件 |
| 位点 | offset / binlog position / LSN | 标识增量消费进度的坐标，MySQL 为 binlog 文件名加偏移量或 GTID，PostgreSQL 为日志序列号（Log Sequence Number，LSN），SQL Server 为 LSN |
| 一致性快照 | consistent snapshot | 全量读取开始时刻的统一数据视图，保证全量数据与增量起点不重不漏 |
| 割接 | cutover | 业务流量从源库切换到目标库的动作 |
| 幂等写入 | idempotent write | 同一批变更重复应用多次，结果与应用一次相同，通常借助主键 upsert 实现 |
| Exactly-once | — | 恰好一次语义，结合事务与状态管理，保证变更不重不漏地应用 |
| 复制槽 | replication slot | PostgreSQL 中记录消费位点并防止服务端过早丢弃 WAL 的服务端对象 |
| 逻辑复制 | logical replication | PostgreSQL 原生的基于发布订阅模型、以逻辑格式传输变更的复制机制 |
| 模式演进 | schema evolution | 同步过程中自动处理源端 DDL 变更（加列、改列等）的能力 |
| 数据脱敏 | data masking | 生产数据进入测试环境前对敏感字段做不可逆变换或遮蔽 |

# 第 2 章 四大需求与方案选型

本章把四类迁移需求映射到工具能力，给出业界主流开源方案的全景对比与选型决策方法。完成本章学习后，读者应能为自己的场景选定一个主方案与一个备选方案。

## 2.1 四大需求与能力对照

四类需求对工具的要求并不相同，先将其拆解为可核对的能力项。

1. **异构迁移**：工具需要在读写两端提供不同引擎的连接器，并内置类型映射规则。仅支持单一引擎源端的工具（如 Canal 仅支持 MySQL 系 binlog）不能满足该需求。
2. **部分表迁移**：工具需要支持按表名列表或通配符、正则选择同步范围，同时能在目标端只创建选中的表。
3. **条件筛选迁移**：工具需要支持在抽取阶段附加 `WHERE` 条件或自定义 SQL，并保证筛选条件与分片并行逻辑正确组合。
4. **实时迁移**：工具需要具备变更日志捕获能力或可接入 CDC 事件流，并支持位点管理与故障恢复。

## 2.2 业界开源方案全景

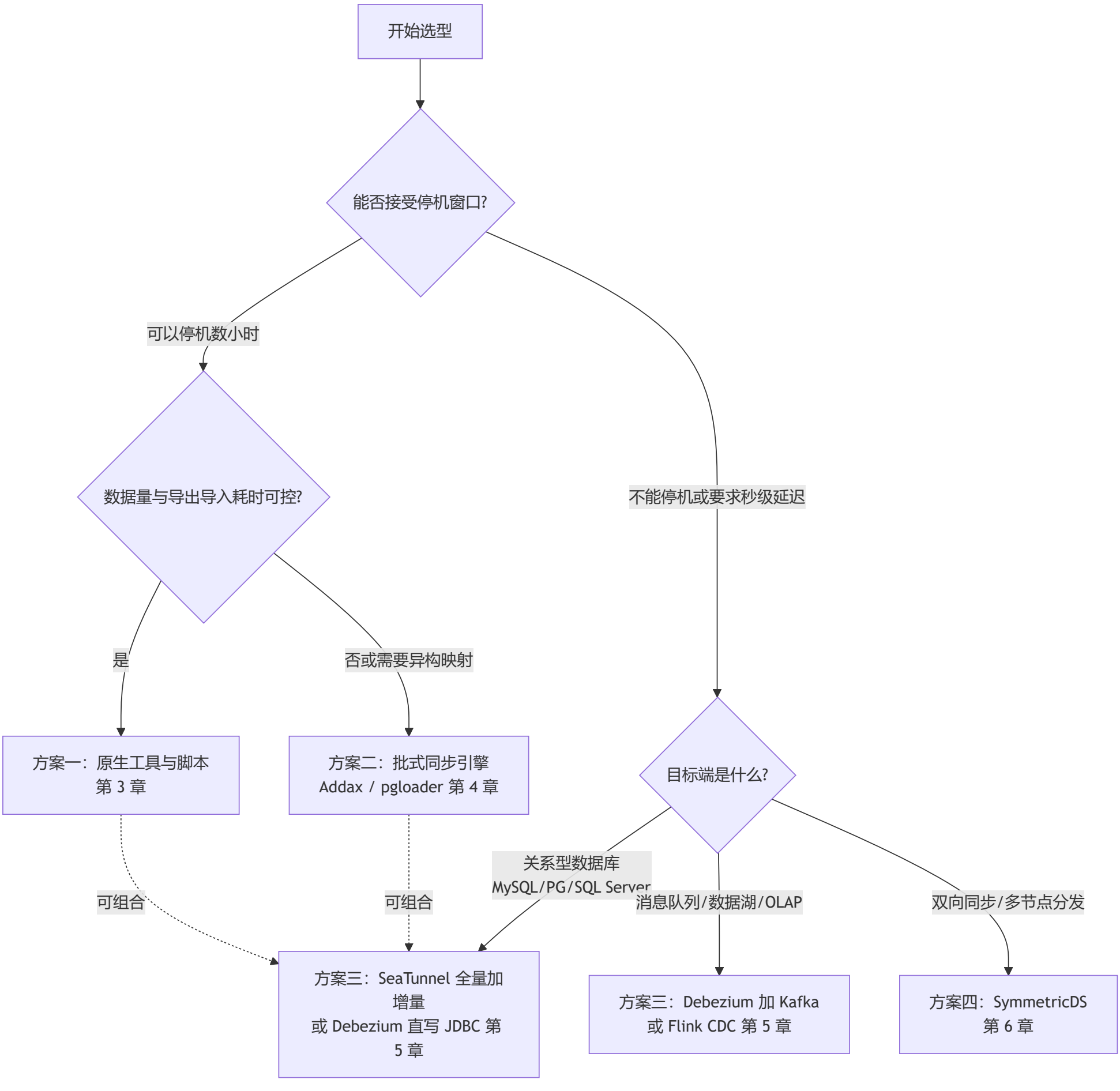
下表汇总了经过生产验证的主流开源工具（版本为 2026-10-25 核实的最新稳定版本）。

| 工具 | 许可证 | 定位 | 异构 | 部分表 | 条件筛选 | 实时 |
| --- | --- | --- | --- | --- | --- | --- |
| mysqldump / mydumper+myloader | GPL | MySQL 逻辑导出导入 | 否（MySQL 系） | 支持 | 支持（`--where`） | 否 |
| pg_dump / pg_restore | PostgreSQL Licence | PostgreSQL 逻辑导出导入 | 否（PostgreSQL 系） | 支持（`-t`） | 部分（整表级，行级需 `\copy`） | 否 |
| bcp / BULK INSERT | 随 SQL Server 免费提供 | SQL Server 批量进出 | 否（SQL Server） | 支持 | 支持（`queryout` 查询导出） | 否 |
| pgloader | PostgreSQL Licence | MySQL、SQLite、SQL Server 一步迁移到 PostgreSQL | 是（仅目标是 PostgreSQL） | 支持（`INCLUDING ONLY TABLES MATCHING`） | 否（不支持行级筛选） | 否 |
| Addax（DataX 后继） | Apache-2.0 | 批式 ETL，30 余种读写插件 | 是 | 支持 | 支持（`where`、`querySql`） | 否 |
| Apache SeaTunnel | Apache-2.0 | 一体化数据集成引擎，支持全量加增量 CDC | 是 | 支持（表列表与正则） | 支持（`query`、Transform） | 是 |
| Debezium（Kafka Connect） | Apache-2.0 | CDC 平台，变更事件进 Kafka | 是（源端 10 余种数据库） | 支持（`table.include.list`） | 支持（快照条件与 SMT 过滤） | 是 |
| Debezium Server | Apache-2.0 | 免 Kafka 的独立 CDC 运行时，可直写 JDBC 目标 | 是 | 支持 | 支持 | 是 |
| Apache Flink CDC | Apache-2.0 | 基于 Flink 的 CDC 集成，YAML 描述流水线 | 是（Pipeline 源端含 MySQL、PostgreSQL、SQL Server、Oracle） | 支持 | 支持（Transform 过滤） | 是（分析型目标端为主） |
| Canal | Apache-2.0 | MySQL binlog 增量订阅与投递 | 否（仅 MySQL 系源端） | 支持（正则过滤） | 支持（客户端侧过滤） | 是 |
| SymmetricDS 社区版 | AGPL-3.0 | 触发器捕获的双向异构复制 | 是（80 余种数据端） | 支持 | 支持（路由与条件表达式） | 是（近实时） |

除上述开源方案外，各云厂商的 DTS（Data Transmission Service）类托管服务覆盖同样能力，属于商业方案，本教程不展开，仅在选型时作为对照。

## 2.3 方案选型决策树

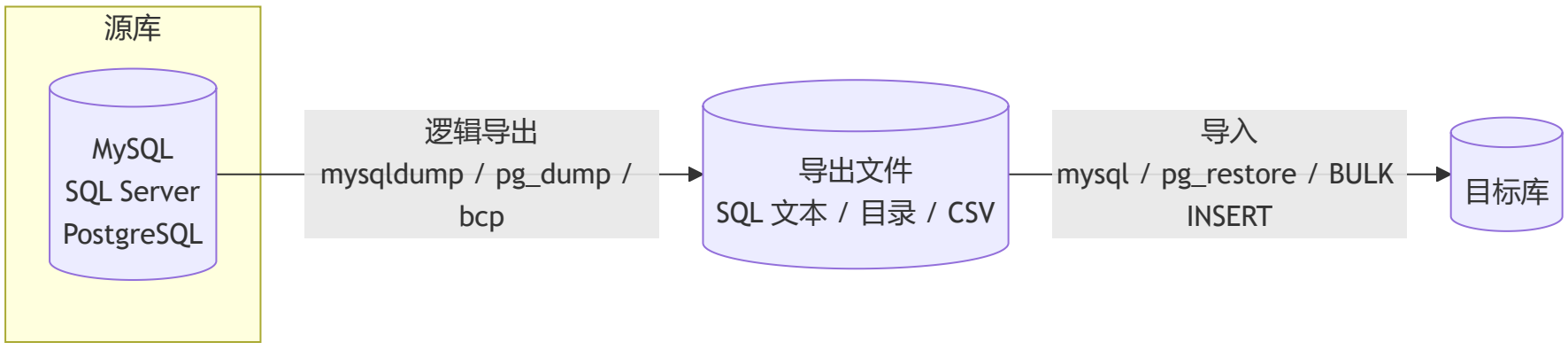
选型的四个关键输入是：停机预算、数据量、是否异构、是否要求持续同步。下图给出推荐路径。



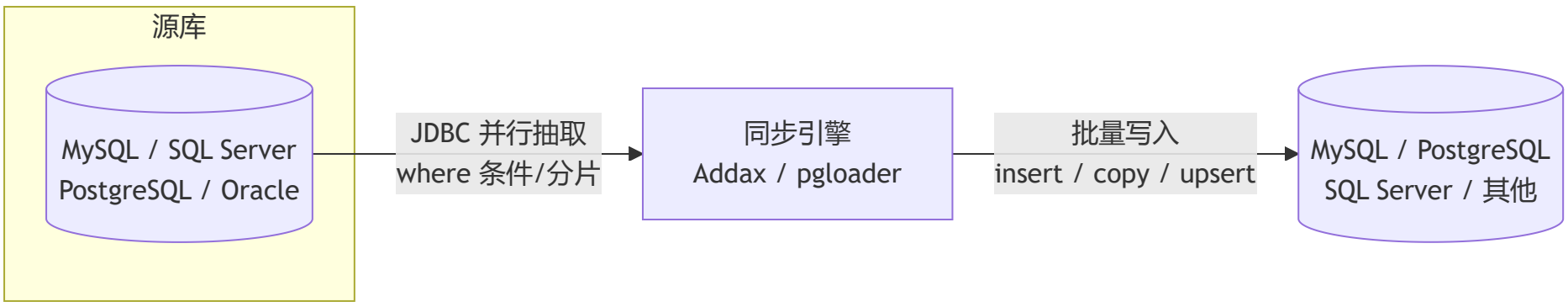

两条虚线表示生产级不停服迁移的标准组合：全量用方案一或方案二搬运，增量用方案三追平。

## 2.4 三大主力方案架构速览

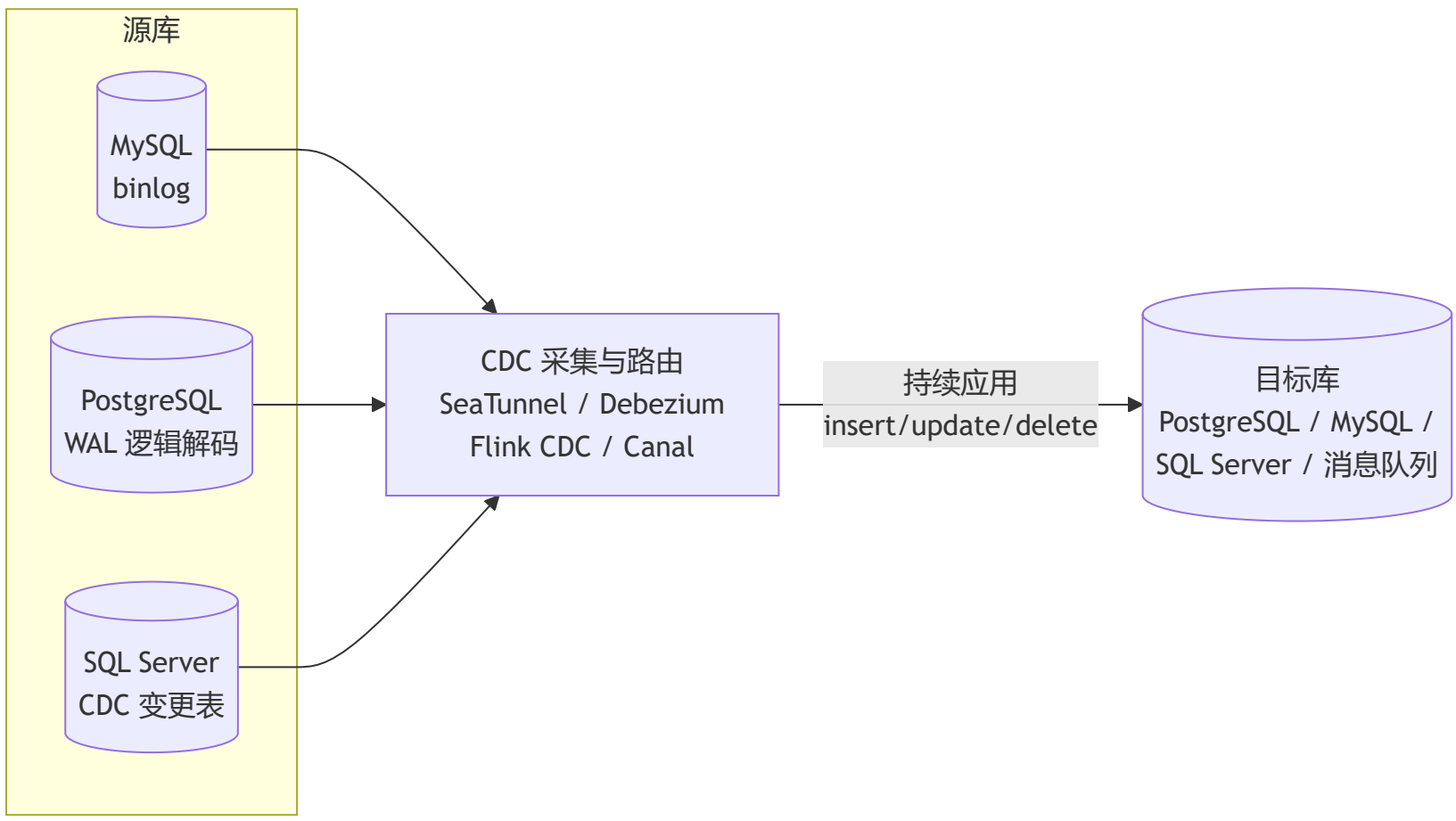
**方案一：原生工具与脚本。** 利用数据库自带工具完成导出导入，适合可停机、规模可控、或同构搬迁的场景。优点是零额外组件、行为透明；缺点是异构类型转换需人工处理、无增量能力（PostgreSQL 原生逻辑复制除外）。




**方案二：批式同步引擎。** 以 Addax 与 pgloader 为代表。Addax 走通用 ETL 框架路线，读写插件异构组合；pgloader 专攻「一键迁入 PostgreSQL」。适合大表并行搬迁、周期性批量同步、以及生产向测试的分发。




**方案三：实时 CDC 同步。** 以 SeaTunnel、Debezium、Flink CDC、Canal 为代表。核心是捕获事务日志并持续应用到目标端，是实时迁移与不停服割接的基础。生产部署通常为「全量快照加增量流式」一体化形态。




**方案四：双向持续复制。** 以 SymmetricDS 社区版为代表，基于触发器捕获，支持多主双向、按节点路由与子集同步，适合门店类边缘节点汇聚与长期异构镜像；因许可证为 AGPL-3.0 且触发器对源库有一定开销，定位与前三条路径不同。

### 2.5 选型建议汇总

下表给出典型场景的推荐组合，理由随各方案章节展开。

| 场景 | 推荐主方案 | 说明 |
| --- | --- | --- |
| 可停机数小时，同构搬迁，数据量中等 | 方案一 | 原生工具最透明，风险最低 |
| 可停机，但表多量大、需要并行与筛选 | 方案二（Addax） | 分片并行与条件筛选能力完备 |
| 迁移目标是 PostgreSQL，且结构以工具自动转换为宜 | 方案二（pgloader） | 类型映射与建表自动化程度高 |
| 生产向测试分发部分表并脱敏 | 方案二为主，方案三辅助 | 周期性子集搬迁，Transform 做脱敏 |
| 不停服异构搬迁，目标为关系库 | 方案三（SeaTunnel 或 Debezium Server JDBC） | 全量加增量一体化，秒级延迟 |
| 不停服同步到消息队列、数据湖、OLAP | 方案三（Debezium 加 Kafka 或 Flink CDC） | 事件流生态成熟 |
| 长期双向同步、多节点分发 | 方案四（SymmetricDS） | 社区版即可覆盖单向与双向场景 |

# 第二篇 方案篇

## 第 3 章 方案一：原生工具与脚本

本章讲解如何仅用数据库自带的工具完成迁移，包括 MySQL 系的 mysqldump 与 mydumper、PostgreSQL 系的 pg_dump 与 pg_restore、SQL Server 的 bcp 与备份恢复，以及 PostgreSQL 原生逻辑复制实现的近零停机迁移。该方案的适用判据是：允许出现停机窗口、希望减少额外组件、或需要精确到行的筛选控制。

## 3.1 入门：跑通第一条迁移链路

### 3.1.1 MySQL 到 MySQL：mysqldump

`mysqldump` 是 MySQL 官方逻辑导出工具，将表结构与数据导出为 SQL 文本，再用 `mysql` 客户端导入。导出命令如下。

```
mysqldump -h 10.0.0.11 -P 3306 -u migrator -p \
  --single-transaction \
  --source-data=2 \
  --databases shop \
  > shop_dump.sql
```

参数说明：`--single-transaction` 使 InnoDB 表的导出基于单个一致性快照完成，全程不阻塞业务写入；`--source-data=2` 把导出时刻的 binlog 位点以注释形式记录进转储文件（MySQL 8.0.26 之前的版本使用等价旧参数 `--master-data=2`），该位点供后续增量同步使用；`--databases shop` 指定导出的库。只导出部分表时，将库名与表名依次列出即可。

```
mysqldump -h 10.0.0.11 -u migrator -p \
  --single-transaction \
  shop orders order_items > part_dump.sql
```

在目标库导入：

```
mysql -h 10.0.0.21 -u migrator -p < shop_dump.sql
```

预期结果为导入完成后目标库出现同名库表且行数一致。需要注意：mysqldump 默认会写入 `DROP TABLE` 语句，向已有数据的测试库导入时会先删后建。

### 3.1.2 MySQL 到 MySQL：mydumper 与 myloader

`mydumper` 是社区维护的多线程逻辑导出工具（配套导入工具为 `myloader`），相比单线程的 mysqldump 能显著缩短大库的导出导入时间。导出示例：

```
mydumper -h 10.0.0.11 -u migrator -p 'Passw0rd' \
  -B shop \
  -T shop.orders,shop.order_items \
  --where "created_at >= '2025-01-01'" \
  -r 200000 \
  -t 8 \
  -o /data/dump/shop
```

参数说明：`-B` 指定数据库；`-T` 指定表列表，表名必须带库名前缀（如 `shop.orders`），用于部分表迁移；`--where` 对本次导出的所有表追加行级筛选条件；`-r` 指定每片数据的行数，用于大表切分并行；`-t` 指定线程数；`-o` 指定输出目录。若需要对不同表施加不同的筛选条件，可使用配置文件中按 `[库名.表名]` 分节的 `where` 与 `limit` 条目，具体写法以官方文档为准。

导入示例：

```
myloader -h 10.0.0.21 -u migrator -p 'Passw0rd' \
  -d /data/dump/shop \
  -o
```

参数说明：`-d` 指定备份目录；`-o` 表示目标表已存在时先删除再导入。myloader 同样支持 `-T` 与 `--regex` 过滤，实现只恢复部分表。

### 3.1.3 PostgreSQL 到 PostgreSQL：pg_dump 与 pg_restore

`pg_dump` 支持并行导出与并行恢复，目录格式（`-Fd`）是并行能力的前提。

```
pg_dump -h 10.0.0.11 -U migrator \
  -d shop \
  -Fd -j 4 \
  -f /data/dump/shop_dir
```

参数说明：`-Fd` 指定目录格式；`-j 4` 指定 4 个并行任务；`-f` 指定输出目录。只导出部分表时追加 `-t orders -t order_items`（表级选择），只导出某个模式时使用 `-n schema_name`。

恢复到新库：

```
createdb -h 10.0.0.21 -U migrator shop_new
pg_restore -h 10.0.0.21 -U migrator \
  -d shop_new \
  -j 4 \
  /data/dump/shop_dir
```

pg_dump 不支持行级筛选。需要按条件导出部分行时，使用客户端 `\copy` 命令把任意查询结果导出为文件，再在目标端导入：

```
-- 源端 psql 中执行，导出满足条件的行
\copy (SELECT * FROM orders WHERE created_at >= '2025-01-01') TO '/data/orders_2025.csv' WITH (FORMAT csv, HEADER true)
```

```
-- 目标端 psql 中执行，先建好结构再导入
\copy orders FROM '/data/orders_2025.csv' WITH (FORMAT csv, HEADER true)
```

#### 3.1.4 SQL Server 到 SQL Server：备份恢复与 bcp

停机窗口内同构搬迁 SQL Server 最可靠的方式是完整备份加恢复，无任何外部依赖，此处不再展开命令。需要部分表或行级筛选时，使用 `bcp`（bulk copy program，随 SQL Server 客户端工具免费提供）按查询导出：

```
bcp "SELECT * FROM shop.dbo.orders WHERE created_at >= '2025-01-01'" queryout "orders.csv" -c -t"," -S 10.0.0.11 -d shop -U migrator -P "Passw0rd"
```

参数说明：`queryout` 表示导出对象是一条查询而非整表，这是 bcp 实现行级筛选的方式；`-c` 按字符模式导出；`-t","` 指定字段分隔符；`-S` 指定服务器；`-d` 指定数据库。目标端使用 `BULK INSERT` 批量导入：

```
BULK INSERT dbo.orders
FROM 'D:\data\orders.csv'
WITH (
    FORMAT = 'CSV',
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n'
);
```

SQL Server 生态中还有免费的图形化与命令行迁移工具（如 SSMS 导入导出向导、SqlPackage 生成的 BACPAC 包），它们免费但并非开源，本教程仅提示其存在，不作为主线。

### 3.2 异构迁移：用原生工具从 MySQL 迁往 PostgreSQL

原生工具做异构迁移的本质是「结构人工转换、数据经中间格式搬运」。以 MySQL 到 PostgreSQL 为例，标准做法分三步。

第一步，在 PostgreSQL 端按目标类型人工建表。MySQL 与 PostgreSQL 的常见类型对应关系参见第 8 章，此处先给出一张演示表的建表语句对照。

```
-- MySQL 源端结构
CREATE TABLE orders (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  amount DECIMAL(10,2),
  status TINYINT(1),
  created_at DATETIME
);
```

```
-- PostgreSQL 目标端结构
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id INT NOT NULL,
  amount NUMERIC(10,2),
  status BOOLEAN,
  created_at TIMESTAMP
);
```

第二步，从 MySQL 导出中间格式。推荐导出 CSV 而非 SQL 文本，便于目标端使用高效装载命令：

```
mysql -h 10.0.0.11 -u migrator -p --batch --raw -e \
  "SELECT id, user_id, amount, IF(status, 'true','false'), created_at FROM shop.orders WHERE created_at >= '2025-01-01'" \
  shop > orders.csv
```

`--batch` 使输出采用制表符分隔并转义特殊字符，`--raw` 关闭再转义，二者配合可得到适合装载的规整文本；`IF(status, 'true','false')` 把 MySQL 的布尔语义显式转换为 PostgreSQL 可识别的字面量。更简单的替代方式是直接在 MySQL 端使用 `SELECT ... INTO OUTFILE` 生成文件，但该语句受 `secure_file_priv` 服务端配置限制。

第三步，在 PostgreSQL 端使用 `\copy` 或 `COPY` 装载：

```
\copy orders FROM '/data/orders.csv' WITH (FORMAT csv, HEADER false)
```

数据量达到千万行级时，建议先删除或延后创建索引与外键，装载完成后再补建，并使用 `ANALYZE` 刷新统计信息。此流程完全手工可复用性差，数据量大或表多时应改用第 4 章的 Addax 或 pgloader。

### 3.3 进阶：PostgreSQL 原生逻辑复制实现近零停机

PostgreSQL 内置逻辑复制（logical replication）基于发布（publication）与订阅（subscription）模型，能够把源库的行级变更以逻辑格式持续传送到另一个 PostgreSQL 实例，是实现 PostgreSQL 同构不停服迁移的官方途径，无需引入外部组件。

第一步，在源库开启逻辑 WAL 并配置复制资源上限（修改 `postgresql.conf` 后需重启实例）：

```
wal_level = logical
max_replication_slots = 10
max_wal_senders = 10
```

第二步，在源库创建发布。发布只包含 DML 变更，不复制 DDL，也不复制序列（sequence）的当前值：

```
CREATE PUBLICATION pub_shop FOR TABLE orders, order_items;
```

第三步，先完成结构迁移与可选的初始全量（两种方式二选一：让订阅端自动复制初始数据，或先用 pg_dump 的 `--schema-only` 与手工装载建立基线）。创建订阅：

```
CREATE SUBSCRIPTION sub_shop
  CONNECTION 'host=10.0.0.11 port=5432 dbname=shop user=migrator password=Passw0rd'
  PUBLICATION pub_shop
  WITH (copy_data = true);
```

`copy_data = true` 表示订阅创建后自动对发布的表执行初始同步，随后持续应用增量。执行 `UPDATE` 与 `DELETE` 的正常复制要求表具有主键或可用的复制标识（`REPLICA IDENTITY`），无主键表只能可靠复制插入操作。

第四步，追平后割接。割接窗口内需要处理三件逻辑复制不覆盖的事情：在源端停写；同步序列当前值（对自增主键至关重要）；在目标端补建存储过程、触发器与视图等对象。监控订阅状态使用 `pg_stat_subscription` 视图，监控位点堆积使用 `pg_replication_slots` 视图。长期挂起的复制槽会阻止服务端回收 WAL，造成磁盘空间持续增长，迁移完成后务必删除不再使用的槽与订阅。

### 3.4 精通要点：一致性、校验与脚本化

原生工具方案的精确性来自对细节的控制，以下三点决定迁移质量。

**一致性与位点对齐。**全量导出必须与增量起点对齐，否则会出现数据缺失或重复。MySQL 的标准做法是：导出时使用 `--single-transaction` 保证快照一致，同时用 `--source-data=2` 记录 binlog 位点，后续 CDC 从该位点启动。PostgreSQL 逻辑复制的初始同步由订阅机制自动完成对齐，无需人工记录位点。

**数据校验。**迁移后至少执行三个层次的比对：行数比对（两端对同一表执行 `COUNT(*)`，带条件筛选时两端使用同一条件）；聚合比对（对数值列执行 `SUM`、对日期列执行 `MIN` 与 `MAX`）；抽样明细比对（按主键抽样若干行逐列核对）。校验脚本应保存并纳入版本管理，第 11 章给出通用脚本骨架。

**脚本化与幂等重跑。**把导出、导入、校验封装为可重跑的脚本；导入前清理目标表或使用先删后建的约定，保证脚本重复执行结果一致。向测试环境分发的场景尤其适合把该流程做成每日定时任务。

---

# 第 4 章 方案二：批式同步引擎（Addax 与 pgloader）

本章讲解两个互补的批式工具：Addax 是通用 ETL 框架，可自由组合异构读写插件，行级筛选与并行分片能力完备；pgloader 专注把 MySQL、SQLite、SQL Server 一步迁入 PostgreSQL，结构转换自动化程度最高。二者都是开源免费软件。

## 4.1 Addax：DataX 的活跃后继

**背景说明。**DataX 是阿里巴巴开源的离线数据同步框架，在国内生产环境使用广泛，但项目自 2023 年起基本停止维护（官方仓库冻结）。Addax 是其社区维护的活跃后继（fork 演进版），保留 DataX 的作业配置模型并持续修复缺陷、增加插件，最新稳定版为 6.2.0（2026-10-13 发布）。新项目建议直接使用 Addax；已有 DataX 作业的配置结构与 Addax 高度兼容，可以低成本迁移。

**安装。**推荐两种方式。容器方式（命令已核实官方 README）：

```
docker pull quay.io/wgzhao/addax:latest
docker run -ti --rm --name addax \
  quay.io/wgzhao/addax:latest \
  /opt/addax/bin/addax.sh /opt/addax/job/job.json
```

物理机方式使用官方 README 提供的安装脚本或从发布页下载安装包自行部署，Addax 6.x 需要 JDK 17 及以上版本，安装步骤以官方 README 为准。

## 4.2 入门：MySQL 全量同步到 PostgreSQL

Addax 作业是一个 JSON 文件，描述「读什么、写什么、怎么跑」。下例把 MySQL `shop` 库的 `orders` 表全量同步到 PostgreSQL `new_shop` 库的 `orders` 表。

```
{
  "job": {
    "setting": {
      "speed": {
        "channel": 4
      },
      "errorLimit": {
        "record": 0
      }
    },
    "content": [
      {
        "reader": {
          "name": "mysqlreader",
          "parameter": {
            "username": "migrator",
            "password": "Passw0rd",
            "column": ["*"],
            "connection": [
              {
                "table": ["orders"],
                "jdbcUrl": ["jdbc:mysql://10.0.0.11:3306/shop?useSSL=false&serverTimezone=Asia/Shanghai"]
              }
            ]
          }
        },
        "writer": {
          "name": "postgresqlwriter",
          "parameter": {
            "username": "migrator",
            "password": "Passw0rd",
            "column": ["id", "user_id", "amount", "status", "created_at"],
            "connection": [
              {
                "jdbcUrl": "jdbc:postgresql://10.0.0.21:5432/new_shop",
                "table": ["orders"]
              }
            ]
          }
        }
      }
    ]
  }
}
```

结构说明：`job.setting.speed.channel` 定义并发通道数，是吞吐的首要调节项；`errorLimit.record` 定义允许的脏数据条数上限，设为 0 表示出现一条脏数据即失败，迁移场景建议保持 0 以避免静默丢数；`content` 数组中的每个元素是一组读写对。`mysqlreader` 基于 RDBMS Reader 基类实现，`column` 为抽取列集合，`["*"]` 表示全部列；`postgresqlwriter` 的 `column` 必须与目标表列名一致，异构列名不同时在此处按目标顺序填写即可完成列映射。

执行作业：

```
bin/addax.sh job/mysql2pg_orders.json
```

正常结束时命令行输出读写计数统计，`Total Records` 两端数值一致即本次作业成功。

## 4.3 部分表、列裁剪与并行分片

**部分表迁移。**一个作业可配置多组读写对，每组一张表；工程实践中更常用的做法是为每张表生成独立作业文件，便于单独重跑与并发调度。筛选目标库范围时无须额外配置——Addax 不做结构迁移，目标表需要预先建好（可结合第 3 章的结构导出或第 8 章的建表脚本）。

**列裁剪与列排除。**`column` 数组显式列出需要的列即完成列裁剪，还支持常量与表达式；当列很多而只想排除少数列时，使用 `excludeColumn`，该参数仅在 `column` 配置为 `["*"]` 时生效：

```
"column": ["*"],
"excludeColumn": ["password_hash", "id_card_no"]
```

**并行分片。**单表数据量大时，通过 `splitPk` 按指定字段切分数据并行抽取，推荐使用主键：

```
"splitPk": "id"
```

`splitPk` 仅支持整型与 ASCII 字符串类型字段。若不想手工指定，可开启 `autoPk: true`（Addax 3.2.6 引入），引擎将自动探测表的主键或单字段唯一索引作为切分键，已支持的数据库包括 MySQL、PostgreSQL、SQL Server、Oracle、ClickHouse 等。总并发由 `channel` 数与分片数共同决定。

## 4.4 条件筛选迁移

RDBMS 系读插件提供两种行级筛选方式。

**`where` 条件。**追加到自动生成的 `SELECT` 语句之后，与 `table` 配合使用，并且会与 `splitPk` 分片正确组合（引擎在生成每个分片的区间查询时把该条件一并下推）：

```
"where": "created_at >= '2025-01-01' AND status = 1"
```

**`querySql` 自定义查询。**完全接管抽取 SQL，与 `table` 配置互斥，适用于跨表查询、函数加工等场景，也支持用 `@/path/to/file.sql` 从外部文件读取 SQL：

```
"querySql": [
  "SELECT o.id, o.user_id, u.name FROM orders o JOIN users u ON o.user_id = u.id WHERE o.created_at >= '2025-01-01'"
]
```

使用 `querySql` 时分片能力失效（引擎无法安全切分任意查询），大表筛选导出应优先使用 `where` 加 `splitPk` 的组合。

## 4.5 生产向测试分发与脱敏

生产环境向测试环境迁移通常叠加两个要求：只搬部分数据、敏感字段脱敏。Addax 的实现路径如下：表与行级范围用 `column`、`excludeColumn`、`where` 控制；字段变换用内置 transformer 完成。Addax 提供数据转换（transformer）层，支持内置函数与 Groovy 自定义脚本（`dx_groovy`），可在读取后、写入前对字段做掩码、哈希等处理，例如把手机号替换为哈希值、把身份证号截断为前六位。具体 transformer 清单与参数以官方「数据转换」文档为准。

一个典型的测试库分发作业组合为：`where` 限定最近一年的数据，`excludeColumn` 排除敏感列，transformer 对剩余敏感列做哈希，`postgresqlwriter` 或 `mysqlwriter` 以先清空后写入的方式覆盖测试库。

## 4.6 pgloader：一步迁入 PostgreSQL

**定位。**pgloader 是开源命令行工具（PostgreSQL Licence），支持从 MySQL、SQLite、MS SQL Server 等源端把结构与数据一次性迁入 PostgreSQL，自动完成建表、类型转换、并行装载、索引与约束重建。当前稳定系列为 v3.6.x（Common Lisp 实现）；v4 为 Clojure 全面重写版，以单个可执行 JAR 分发、需要 Java 21，官方声明 v4 是 v3 的即插即用替代（接受相同的 `.load` 命令文件语法与命令行参数），撰写本教程时 v4 处于预发布阶段，生产使用前请确认其发布状态。

**安装。**多数发行版可直接安装：Debian 与 Ubuntu 使用 `apt install pgloader`，macOS 使用 `brew install pgloader`。v4 则从官方发布页下载 JAR 后以 `java -jar pgloader.jar` 方式运行。

**整库迁移。**一条命令完成（示例已按官方 README 核实）：

```
pgloader mysql://migrator:Passw0rd@10.0.0.11/shop postgresql://migrator:Passw0rd@10.0.0.21/new_shop
```

**命令文件迁移。**需要控制迁移范围与规则时使用 `.load` 命令文件：

```
LOAD DATABASE
     FROM mysql://migrator:Passw0rd@10.0.0.11:3306/shop
     INTO postgresql://migrator:Passw0rd@10.0.0.21:5432/new_shop

WITH include drop, create tables, create indexes, reset sequences,
     foreign keys, downcase identifiers

CAST type tinyint when (= precision 1) to boolean drop typemod,
     type datetime to timestamptz drop default

INCLUDING ONLY TABLES MATCHING ~/^orders/, ~/^users/
EXCLUDING TABLES MATCHING ~/^tmp_/
;
```

逐项说明：`WITH` 子句控制行为，`include drop` 表示迁移前先删除目标同名对象，`reset sequences` 把自增对应的序列值校准到已迁移数据的最大值，`downcase identifiers` 把标识符统一转为小写（PostgreSQL 未加引号的标识符默认小写，这一步可避免大小写陷阱）；`CAST` 子句改写默认类型映射，示例中把单精度位的 `tinyint(1)` 映射为布尔、`datetime` 映射为带时区时间戳；`INCLUDING ONLY TABLES MATCHING` 与 `EXCLUDING TABLES MATCHING` 分别实现表级白名单与黑名单，匹配对象支持正则。

执行：

```
pgloader shop.load
```

**能力边界。**pgloader 的表级筛选完善，但不提供任意行级 `WHERE` 筛选能力；需要按条件筛选行时，应改用 Addax（`where`、`querySql`）或 SeaTunnel（`query`）。pgloader 不迁移存储过程、触发器与视图等程序性对象，这些对象需要人工改写。

### 4.7 精通要点：性能、幂等与调度

**吞吐调优。**Addax 的吞吐由通道数（`channel`）、切分键（`splitPk` 或 `autoPk`）、批量提交大小与目标端写入模式共同决定。目标端写入侧的 `batchSize` 控制每批提交行数；PostgreSQL 目标端可关注写入插件是否支持 `COPY` 协议模式，`COPY` 的装载吞吐通常显著高于逐条 `INSERT`。源端压力控制通过 `speed` 中的 `byte`、`record` 限速项实现，生产库在线抽取时务必设置限速并避开业务高峰。

**幂等与重跑。**写入插件提供 `preSql` 与 `postSql`（作业前后执行的 SQL）。向测试库覆盖式同步的标准做法是 `preSql` 执行 `TRUNCATE TABLE`，使作业可整体重跑；向生产新库做追加式迁移时使用空表加校验。注意 `TRUNCATE` 会立即提交且不可回滚，目标端如果是共享环境须先确认无他人使用。

**调度与监控。**周期性分发场景将作业纳入调度系统（如 Apache DolphinScheduler，其 DataX 集成同样适用于 Addax 作业），每次运行后校验读写计数；Addax 作业结束输出会包含错误计数，`errorLimit` 触发即任务失败，调度系统据此告警。

---

## 第 5 章 方案三：实时 CDC 同步

本章是全教程的核心章节，讲解基于变更数据捕获的实时迁移方案。覆盖四个工具：SeaTunnel（一体化全量加增量，适合关系库之间）、Debezium（CDC 平台标准实现，Kafka Connect 生态，另有免 Kafka 的 Debezium Server）、Flink CDC（Flink 生态集成，适合流式分析场景）、Canal（轻量 MySQL binlog 订阅）。完成本章学习后，读者应能为不停服迁移搭建一条端到端实时链路。

### 5.1 CDC 原理与源端前置条件

CDC 工具不查询业务表，而是把自己注册为数据库的日志消费者：MySQL 伪装为复制从库读取 binlog，PostgreSQL 通过逻辑解码插槽读取 WAL，SQL Server 读取 CDC 功能生成的变更表。三种日志的开启方式与要求不同，下表汇总源端前置条件（配置项均已于核实日期与官方文档核对）。

| 项目 | MySQL | PostgreSQL | SQL Server |
| --- | --- | --- | --- |
| 日志机制 | binlog | WAL 逻辑解码 | CDC 变更表 |
| 服务端参数 | `log_bin=ON`、`binlog_format=ROW`、`binlog_row_image=FULL` | `wal_level=logical`、`max_replication_slots` 与 `max_wal_senders` 大于消费并发 | 启用 SQL Server Agent 服务 |
| 对象级开启 | 无需（表级无开关） | 创建发布（`CREATE PUBLICATION`） | 数据库级与表级分别执行 `sys.sp_cdc_enable_db`、`sys.sp_cdc_enable_table` |
| 账号权限 | `SELECT`、`RELOAD`、`SHOW DATABASES`、`REPLICATION SLAVE`、`REPLICATION CLIENT` | 具备 `REPLICATION` 属性的角色或 superuser 用户 | 对 CDC 表有 `SELECT` 权限的固定数据库角色成员 |
| 版本要求 | 5.5 至 8.4（SeaTunnel 采集范围）；Debezium 支持 8.0.x、8.4.x、9.x | Debezium 支持 12 至 18；pgoutput 插件要求 10 及以上 | CDC 功能要求 2016 SP1 及以上的 Standard 或 Enterprise 版本 |

MySQL binlog 配置示例（`my.cnf`）：

```
[mysqld]
server-id         = 223344
log_bin           = mysql-bin
binlog_format     = ROW
binlog_row_image  = FULL
binlog_expire_logs_seconds = 604800
```

`binlog_format=ROW` 保证日志记录行级前后镜像而非 SQL 语句，`binlog_row_image=FULL` 保证每次更新记录全部列的值，二者是所有 binlog CDC 工具的通用要求；`binlog_expire_logs_seconds` 控制日志保留时长，必须长于可能的最长故障恢复间隔（示例为 7 天）。创建 CDC 专用账号：

```
CREATE USER 'cdc_user'@'%' IDENTIFIED BY 'Passw0rd';
GRANT SELECT, RELOAD, SHOW DATABASES, REPLICATION SLAVE, REPLICATION CLIENT
  ON *.* TO 'cdc_user'@'%';
FLUSH PRIVILEGES;
```

PostgreSQL 逻辑解码示例：

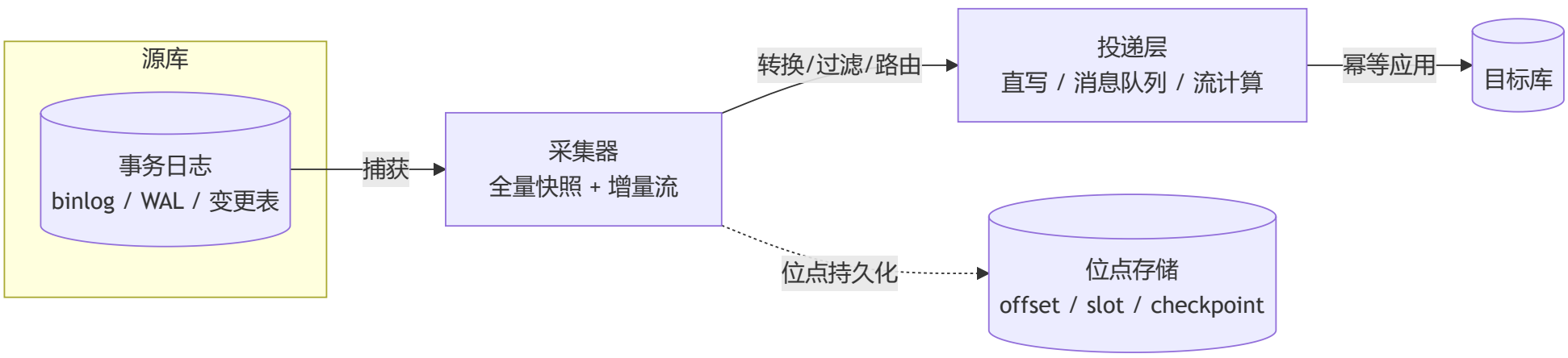
```
-- 参数修改后重启实例
ALTER SYSTEM SET wal_level = 'logical';
-- 建复制专用账号
CREATE ROLE cdc_user WITH LOGIN REPLICATION PASSWORD 'Passw0rd';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO cdc_user;
-- 建发布（CDC 工具连接后按表订阅）
CREATE PUBLICATION pub_all FOR TABLE orders, users;
```

SQL Server CDC 开启示例：

```
-- 数据库级开启（每个库一次）
EXEC sys.sp_cdc_enable_db;
-- 表级开启（每张需要捕获的表一次）
EXEC sys.sp_cdc_enable_table
  @source_schema = N'dbo',
  @source_name   = N'orders',
  @role_name     = NULL;
```

CDC 功能依赖 SQL Server Agent 定期执行捕获作业，Agent 停止则变更停止累积，是运维检查的第一优先项。

实时迁移链路的通用架构如下图。所有工具的差异都在捕获与投递两段的实现方式上。




## 5.2 SeaTunnel：一体化全量加增量

**定位。**Apache SeaTunnel（Apache-2.0 许可证）是分布式数据集成平台，自带 Zeta 引擎（也可运行于 Flink），其 CDC 源端连接器在同一任务内自动完成「一致性快照全量 + 无缝切入 binlog 增量」，目标端通过 JDBC 连接器直写主流关系库，是目前开源方案中覆盖「关系库到关系库实时迁移」最完整的单一工具。本节基于 2.3.13 文档版编写；3.0.0 已发布，升级前请阅读官方迁移说明。

**部署。**从官方下载页获取 `apache-seatunnel-2.3.13-bin.tar.gz` 解压即可，JDBC 驱动放入 `${SEATUNNEL_HOME}/lib/`（Zeta 引擎）。最小单机配置使用自带的 `config/` 示例文件即可运行。

**MySQL 到 PostgreSQL 全量加增量示例。**任务文件 `job/mysql2pg.conf`：

```
env {
  parallelism = 4
  job.mode = "STREAMING"
  checkpoint.interval = 10000
}

source {
  MySQL-CDC {
    url = "jdbc:mysql://10.0.0.11:3306/shop"
    username = "cdc_user"
    password = "Passw0rd"
    table-names = ["shop.orders", "shop.users", "shop.order_items"]
    server-id = "5400-5404"
    server-time-zone = "Asia/Shanghai"
    startup.mode = "initial"
    snapshot.split.size = 8096
  }
}

sink {
  Jdbc {
    url = "jdbc:postgresql://10.0.0.21:5432/new_shop"
    driver = "org.postgresql.Driver"
    user = "migrator"
    password = "Passw0rd"
    generate_sink_sql = true
    database = "new_shop"
    table = "public.${table_name}"
    primary_keys = ["id"]
    enable_upsert = true
    schema_save_mode = "CREATE_SCHEMA_WHEN_NOT_EXIST"
    data_save_mode = "APPEND_DATA"
  }
}
```

逐项说明：`env` 中 `job.mode = "STREAMING"` 表示流式任务（先快照后常驻消费增量），`checkpoint.interval` 是检查点间隔毫秒数，检查点是断点续传与 Exactly-once 的基础；源端 `table-names` 列出同步的表（带库名前缀，实现部分表迁移），`server-id` 建议配置为区间形式供并行读取使用且不得与集群内其他复制消费者冲突，`startup.mode = "initial"` 表示先做一致性快照再进入增量；`snapshot.split.size` 定义快照阶段按主键切分的分片大小（行数）。目标端 `generate_sink_sql = true` 表示由框架依据上游结构自动生成写入 SQL，`table` 支持变量替换（`${table_name}` 等被上游表名替换，从而多表共用一个 sink 配置），`primary_keys` 用于生成 upsert 语义，`enable_upsert = true` 开启按主键合并；`schema_save_mode` 与 `data_save_mode` 控制目标端表结构不存在时的建表行为与已有数据的处理方式（本例为缺表自动创建、数据追加）。数据库若不存在，PostgreSQL 端的库需要预先创建。

执行：

```
$SEATUNNEL_HOME/bin/seatunnel.sh --config job/mysql2pg.conf -e local
```

运行输出包含各阶段进度指标；任务可以 `Ctrl+C` 中断后用相同命令恢复，位点由检查点文件接管。

**正则批量选表。**表数量多时用 `table-pattern` 与 `database-pattern` 正则替代逐一列举（两者与 `table-names` 互斥）：

```
database-pattern = "shop.*"
table-pattern = "shop.*\\.t_.*"
```

**SQL Server 与 PostgreSQL 作为源端。**分别使用 `SqlServer-CDC` 与 `PostgreSQL-CDC` 源端连接器，配置项结构与 MySQL-CDC 一致（`table-names`、`startup.mode` 等）。SQL Server 源端要求先完成 5.1 节的数据库与表级 CDC 开启；PostgreSQL 源端要求 `wal_level=logical` 并具备复制权限，工具将自动创建并管理复制槽。

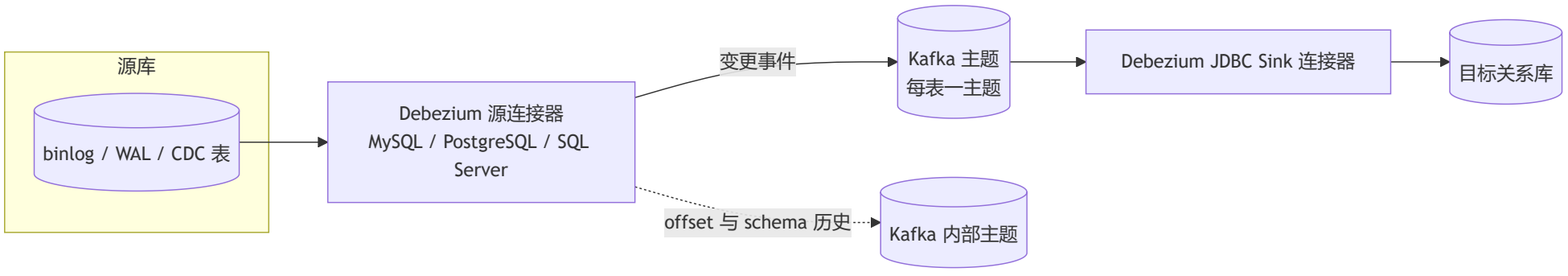
**结构变更同步。**MySQL-CDC 源端设置 `schema-changes.enabled = true` 后，源端加列、删列、改列名、改列类型会在目标端自动执行（当前支持范围以官方文档列出的操作类型为准）。生产环境更稳妥的做法仍是迁移窗口内冻结 DDL，窗口外走常规发布流程并同步评审目标端结构。

**限速与性能。**`env` 中可设置 `read_limit.bytes_per_second` 与 `read_limit.rows_per_second` 限制源端抽取速率；目标端 PostgreSQL 支持开启 `use_copy_statement = true` 走 `COPY FROM STDIN` 装载，批量初始化阶段可显著提速；`batch_size` 控制 JDBC 批量提交行数。 Exactly-once 语义通过 `is_exactly_once = true` 配合 XA 数据源开启，需要目标数据库支持 XA 事务（PostgreSQL 需将 `max_prepared_transactions` 设为大于 0 并重启，MySQL 需 8.0.29 及以上并为账号授予 `XA_RECOVER_ADMIN`）。

## 5.3 Debezium：CDC 平台标准实现

**定位。**Debezium（Apache-2.0 许可证）是基于 Kafka Connect 的分布式 CDC 平台，源端连接器覆盖 MySQL、PostgreSQL、SQL Server、Oracle、MongoDB 等 10 余种数据库，是事实上的 CDC 事件标准实现。其运行时形态有两种：常规形态部署于 Kafka Connect 集群；Debezium Server 为独立运行时，无需 Kafka，可直接把变更写入 JDBC 目标库、Redis、Kinesis 等。

**架构**（Kafka Connect 形态）：




**部署。**下载 Kafka 并启动 Kafka Connect（分布式模式），将 Debezium 源连接器包与 JDBC Sink 连接器包（Debezium 官方提供，或 Confluent 兼容实现）解压至 `plugin.path` 指向的目录，目标库的 JDBC 驱动一并放入，重启 Connect 后即可注册连接器。Debezium 3.7 与 Kafka 4.x 系列配套，版本兼容关系以官方 Release Overview 为准。

**MySQL 源连接器注册示例**（向 Connect REST 接口提交）：

```
{
  "name": "mysql-source-shop",
  "config": {
    "connector.class": "io.debezium.connector.mysql.MySqlConnector",
    "database.hostname": "10.0.0.11",
    "database.port": "3306",
    "database.user": "cdc_user",
    "database.password": "Passw0rd",
    "topic.prefix": "shop-prod",
    "database.include.list": "shop",
    "table.include.list": "shop.orders,shop.users",
    "snapshot.mode": "initial",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter"
  }
}
```

逐项说明：`topic.prefix` 为该连接器产生的所有 Kafka 主题加前缀（Debezium 2.0 起替代旧参数 `database.server.name`）；`table.include.list` 定义捕获的表清单，实现部分表迁移，省略则捕获库中全部非系统表；`snapshot.mode = "initial"` 表示首次启动执行一致性快照随后进入流式。提交方式：

```
curl -X POST -H "Content-Type: application/json" \
  --data @mysql-source-shop.json \
  http://localhost:8083/connectors
```

**JDBC Sink 连接器示例**（写入 PostgreSQL）：

```
{
  "name": "pg-sink-shop",
  "config": {
    "connector.class": "io.debezium.connector.jdbc.JdbcSinkConnector",
    "topics": "shop-prod.shop.orders,shop-prod.shop.users",
    "connection.url": "jdbc:postgresql://10.0.0.21:5432/new_shop",
    "connection.user": "migrator",
    "connection.password": "Passw0rd",
    "insert.mode": "upsert",
    "delete.enabled": "true",
    "primary.key.mode": "record_key",
    "schema.evolution": "basic"
  }
}
```

逐项说明：`insert.mode = "upsert"` 使重复事件按主键幂等合并；`delete.enabled = "true"` 同步删除操作；`primary.key.mode = "record_key"` 以变更事件的主键字段作为目标端主键依据；`schema.evolution = "basic"` 允许目标端自动加列等基础结构演进。目标库驱动 JAR 需预先放入 Connect 插件目录，官方分发包不内置各数据库驱动。

**行级筛选。**Debezium 在两个层面支持条件筛选：其一，全量快照阶段使用 `snapshot.select.statement.overrides.<表名>` 属性为指定表的自定义快照提供带 `WHERE` 的 `SELECT` 语句；其二，增量快照与补充快照通过信号表（signaling table）的 `additional-conditions` 条目附加过滤条件。流式阶段的行级过滤可使用 Kafka Connect SMT（单消息转换）或下游消费逻辑实现。

**Debezium Server 免 Kafka 直写。**当目标只是另一个数据库、不希望引入 Kafka 时，部署 Debezium Server（独立 Java 进程）。核心配置文件 `conf/application.properties` 示例（属性名已按官方示例核实）：

```
debezium.source.connector.class=io.debezium.connector.postgresql.PostgresConnector
debezium.source.offset.storage.file.filename=data/offsets.dat
debezium.source.topic.prefix=shop-prod
debezium.source.database.hostname=10.0.0.11
debezium.source.database.port=5432
debezium.source.database.user=cdc_user
debezium.source.database.password=Passw0rd
debezium.source.database.dbname=shop
debezium.source.table.include.list=public.orders,public.users

debezium.sink.type=jdbc
debezium.sink.jdbc.connection.url=jdbc:postgresql://10.0.0.21:5432/new_shop
debezium.sink.jdbc.connection.username=migrator
debezium.sink.jdbc.connection.password=Passw0rd
debezium.sink.jdbc.insert.mode=upsert
debezium.sink.jdbc.primary.key.mode=record_key
debezium.sink.jdbc.schema.evolution=basic
debezium.sink.jdbc.delete.enabled=true
```

Debezium Server 的 JDBC Sink 依赖事件扁平化转换（`ExtractNewRecordState` SMT）将变更事件还原为行状态，官方建议在源连接器配置中启用该转换；目标库驱动 JAR 需手工放入 `lib/` 目录。该形态将位点存储退化为本地文件（示例中的 `offsets.dat`），生产环境应将数据目录纳入持久化与备份，Kafka 形态则天然具备位点的高可用。

## 5.4 Flink CDC：流计算生态的 CDC 集成

**定位。**Apache Flink CDC 把 CDC 能力与 Apache Flink 流计算引擎结合，提供两类使用方式：YAML 流水线（Pipeline）以整库同步为模型，配置极简；Flink SQL 方式以表为单位，可与任意 Flink SQL 作业组合。最新稳定版 3.7.0（2026-10-10 发布）支持 Flink 1.20.x 与 2.2.x，要求 JDK 11 及以上。

**Pipeline 源端与目标端现状**（以官方连接器总览为准）：Pipeline 源端连接器包括 MySQL、PostgreSQL、Oracle、SQL Server 与 Fluss；Pipeline 目标端连接器包括 StarRocks、Doris、Paimon、Kafka、Elasticsearch、OceanBase、MaxCompute、Iceberg、Hudi 与 Fluss。需要特别注意：**Pipeline 形态当前不提供通用 JDBC 目标端连接器**，因此「MySQL 或 SQL Server 实时迁移到 PostgreSQL 或 MySQL」不直接适用 YAML 流水线，而应使用下述 Flink SQL 方式，或改用 SeaTunnel、Debezium Server。

**Flink SQL 示例：MySQL 实时同步到 PostgreSQL。**在 Flink SQL 客户端中定义 CDC 源表与 JDBC 目标表并建立插入查询：

```
-- 源表：MySQL CDC 连接器
CREATE TABLE orders_mysql (
  id BIGINT,
  user_id INT,
  amount DECIMAL(10, 2),
  status INT,
  created_at TIMESTAMP(3),
  PRIMARY KEY (id) NOT ENFORCED
) WITH (
  'connector' = 'mysql-cdc',
  'hostname' = '10.0.0.11',
  'port' = '3306',
  'username' = 'cdc_user',
  'password' = 'Passw0rd',
  'database-name' = 'shop',
  'table-name' = 'orders',
  'scan.startup.mode' = 'initial'
);

-- 目标表：JDBC 连接器，upsert 语义由主键驱动
CREATE TABLE orders_pg (
  id BIGINT,
  user_id INT,
  amount DECIMAL(10, 2),
  status INT,
  created_at TIMESTAMP(3),
  PRIMARY KEY (id) NOT ENFORCED
) WITH (
  'connector' = 'jdbc',
  'url' = 'jdbc:postgresql://10.0.0.21:5432/new_shop',
  'table-name' = 'orders',
  'username' = 'migrator',
  'password' = 'Passw0rd'
);

INSERT INTO orders_pg SELECT id, user_id, amount, status, created_at FROM orders_mysql;
```

逐项说明：`'scan.startup.mode' = 'initial'` 表示先做全量快照再消费 binlog；源表与目标表都声明主键（`NOT ENFORCED` 表示引擎不校验约束），JDBC 连接器据此把变更写为 upsert。该方式逐表配置、灵活度最高，适合需要流式加工（过滤、关联、脱敏）的同步；整库多表场景配置成本高于 SeaTunnel。

**Pipeline YAML 示例：MySQL 整库同步到 Kafka**（展示 YAML 模型，用于分析与归档场景）：

```
source:
  type: mysql
  hostname: 10.0.0.11
  port: 3306
  username: cdc_user
  password: Passw0rd
  tables: shop.\.*
  server-id: 5400-5404

sink:
  type: kafka
  properties:
    bootstrap.servers: 10.0.0.31:9092
    value.format: debezium-json

route:
  - source-table: shop.\.*
    sink-table: kafka-shop.\.*

pipeline:
  name: shop-to-kafka
  parallelism: 2
```

`tables` 与 `route` 中的正则实现整库选表与库级路由；`transform` 块可定义投影、过滤与字段变换，实现行级筛选。提交命令为 `$FLINK_CDC_HOME/bin/flink-cdc.sh <yaml 文件>`。

## 5.5 Canal：轻量 MySQL binlog 订阅

**定位。**Canal（Apache-2.0 许可证）是阿里巴巴开源的 MySQL binlog 增量订阅组件，把自己伪装为 MySQL 从库拉取并解析 binlog，支持把变更投递到 TCP 客户端、Kafka、RocketMQ 与 Pulsar。它只支持 MySQL 系源端（1.1.8 已适配 MySQL 8.4、MariaDB 11 等），异构目标的落地需要自行实现消费端或使用其 adapter 模块，因此在本教程中的定位是「消息投递型 CDC 的轻量选择」，适合已有 Kafka 基础设施且源端固定为 MySQL 的团队。

**部署要点。**源端配置与 5.1 节相同（ROW 格式、专用账号）。`conf/example/instance.properties` 关键配置（节选，完整项以官方 Wiki 为准）：

```
canal.instance.master.address=10.0.0.11:3306
canal.instance.dbUsername=cdc_user
canal.instance.dbPassword=Passw0rd
canal.instance.filter.regex=shop\\..*
canal.mq.topic=shop-binlog
```

`canal.instance.filter.regex` 以正则实现库表级过滤；投递到 Kafka 时按上述 `canal.mq.topic` 指定主题。消费端从 Kafka 拉取变更后自行写入目标库，写入逻辑必须实现幂等（按主键 upsert）并维护位点。1.1.8 版本提供 canal-admin 管理界面与 Helm 部署支持。

### 5.6 实时迁移的工程要点：位点、幂等与监控

**位点管理。**增量消费的位点（binlog 位点、GTID、LSN、复制槽）必须持久化。位点落后于日志保留期时（MySQL binlog 已被清理、PostgreSQL 复制槽所需 WAL 已被回收、SQL Server 变更表被清理作业清除），增量将无法恢复，只能重新执行全量。因此：日志保留期应显著长于预期最长停机恢复时间；PostgreSQL 要监控 `pg_replication_slots` 中槽的 `confirmed_flush_lsn` 与 WAL 目录增长；SQL Server 要确认 CDC 清理作业的保留窗口。

**幂等写入。**所有 CDC 链路的目标端写入都应按主键 upsert，理由有二：位点提交与事件应用之间不是原子操作，故障重启后可能重复消费同一段事件；快照与流式衔接阶段存在重叠窗口。未启用 upsert 的 append 模式仅适合日志型、允许重复的目标。

**延迟监控。**实时迁移的质量指标是端到端延迟。Debezium 暴露 `MilliSecondsBehindSource` 等指标（经 JMX 与 Kafka Connect 指标接口），SeaTunnel 与 Flink CDC 的检查点与队列积压指标可反映追平进度。割接前应以「延迟稳定低于阈值且持续一定时长」作为放行条件，具体阈值由业务对一致性的要求决定。

### 5.7 四工具对比与选型

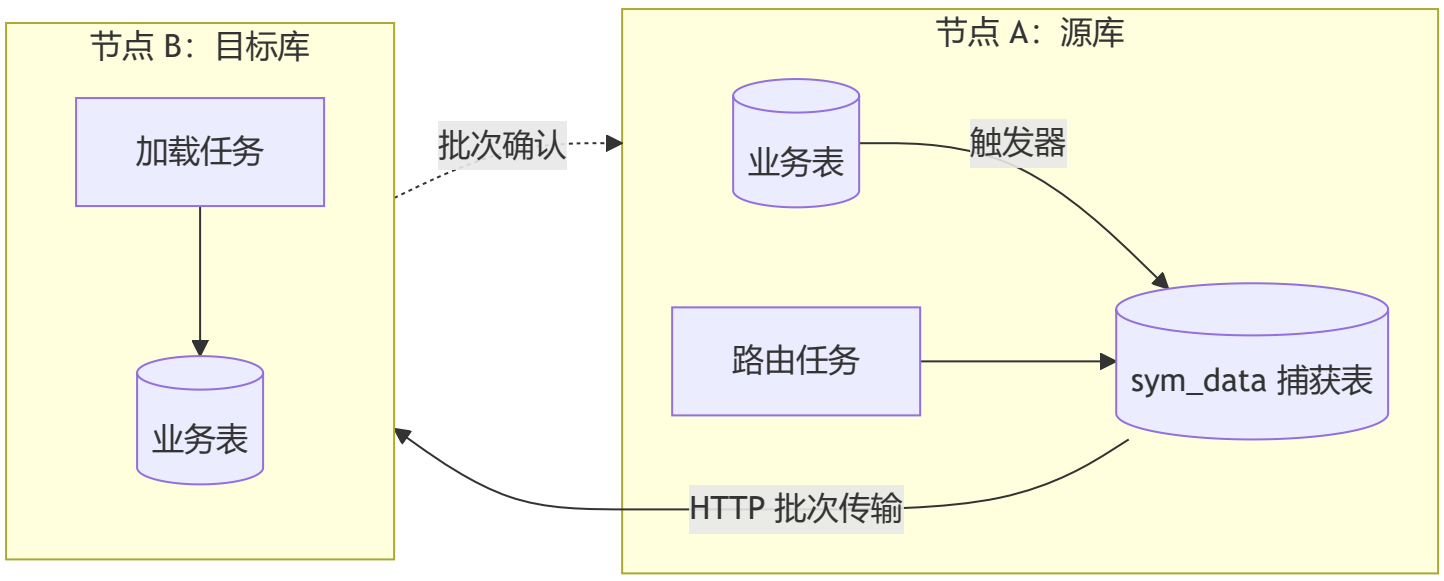
| 维度 | SeaTunnel | Debezium + Kafka | Debezium Server | Flink CDC | Canal |
|---|---|---|---|---|---|
| 运行时复杂度 | 单引擎，低 | 需 Kafka 集群，中 | 单进程，低 | 需 Flink 集群，中 | 单进程加可选 MQ，低 |
| 关系库直写目标 | 原生支持（JDBC Sink） | 经 JDBC Sink 连接器 | 原生支持（JDBC Sink） | 经 Flink SQL JDBC 连接器 | 需自行开发消费端 |
| 全量加增量一体化 | 原生 | 原生（初始快照） | 原生 | 原生 | 仅增量（全量需另接工具） |
| 结构演进支持 | 支持（MySQL 源端） | schema history 机制 | 支持 | 框架级支持 | 需人工处理 |
| 适用场景 | 关系库间实时迁移首选 | 已有 Kafka 且多下游订阅 | 免 Kafka 的直写迁移 | 需流式加工或入湖入仓 | 轻量 MySQL 增量投递 |

## 第 6 章 方案四：双向复制与持续同步（SymmetricDS）

本章介绍 SymmetricDS 社区版。它与前三条路径的定位差异在于：面向长期存在的多节点同步网络，而非一次性搬迁；捕获方式为数据库触发器，因此对源库 schema 有少量侵入（自动创建以 `sym_` 为前缀的配置与队列表），但不依赖事务日志，因而对数据库版本与托管环境的限制最少。社区版许可证为 AGPL-3.0，日志级捕获与部分运维能力属于商业 Pro 版，评估时需要区分。

### 6.1 原理与架构

SymmetricDS 是开源的数据库与文件同步软件（社区版由 JumpMind 赞助维护，最新版本系列为 3.16），支持多主双向复制、按条件过滤同步与数据变换，通过 HTTP/HTTPS 传输批次，可穿越防火墙并容忍长时间断网后补传。工作流程为：在受管表上创建触发器，增删改发生时写入捕获表 `sym_data`；路由任务按规则把数据组织为批次写入 `sym_outgoing_batch`；目标节点拉取批次并在目标端应用。初始全量通过「初始加载」（initial load）机制完成，与增量共用同一套批次与确认机制。




## 6.2 入门：两个节点的单向同步

以 MySQL（源）到 PostgreSQL（目标）为例。下载社区版解压即为运行目录，每个节点一份 `engines/` 下的 properties 配置。

源端节点配置 `engines/corp.properties`（节选）：

```
engine.name=corp
node.group.id=corp
external.id=corp-01
db.url=jdbc:mysql://10.0.0.11:3306/shop?useSSL=false
db.user=migrator
db.password=Passw0rd
registration.url=
sync.url=http://10.0.0.11:31415/sync/corp-01
```

目标端节点配置 `engines/store.properties`：

```
engine.name=store
node.group.id=store
external.id=store-01
db.url=jdbc:postgresql://10.0.0.21:5432/new_shop
db.user=migrator
db.password=Passw0rd
registration.url=http://10.0.0.11:31415/sync/corp-01
sync.url=http://10.0.0.21:31415/sync/store-01
```

逐项说明：`node.group.id` 定义节点组，同步方向按组配置；`external.id` 为节点唯一标识；`registration.url` 指向负责注册的节点，目标节点首次启动将自动注册并拉取配置；`sync.url` 为本节点对外服务地址，默认端口 31415。

启动两侧节点后，在源端注册同步规则（通过 SymmetricDS 自动创建的配置表）：

```
-- 源端执行：定义节点组间的同步方向
INSERT INTO sym_node_group (node_group_id) VALUES ('corp'), ('store');
INSERT INTO sym_node_group_link (source_node_group, target_node_group, data_event_action)
  VALUES ('corp', 'store', 'W');

-- 定义路由与触发器（示例：orders 表单向同步）
INSERT INTO sym_router (router_id, source_node_group, target_node_group, router_type)
  VALUES ('corp_to_store', 'corp', 'store', 'default');
INSERT INTO sym_trigger (trigger_id, source_catalog, source_schema, source_table_name, channel_id,
  sync_on_insert, sync_on_update, sync_on_delete, last_update_time, create_time)
  VALUES ('trg_orders', 'shop', NULL, 'orders', 'default', 1, 1, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
INSERT INTO sym_trigger_router (trigger_id, router_id, initial_load_order, last_update_time, create_time)
  VALUES ('trg_orders', 'corp_to_store', 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

执行以下命令让配置生效并触发初始加载（`initial_load_order` 决定多表初始加载顺序）：

```
bin/symadmin --engine corp send-initial-load
```

完成后目标端将获得 `orders` 表的全量数据，此后源端的增删改以近实时批次持续到达。具体表结构与命令以官方 3.16 User Guide 为准，上述示例仅演示配置模型。

### 6.3 筛选、子集同步与变换

SymmetricDS 的筛选能力落在三个层次，正好覆盖本教程的四类需求。表级：`sym_trigger` 按表注册捕获，未注册的表不参与同步。行级：路由器（`sym_router`）支持按条件表达式决定某行发往哪些目标节点，`sym_trigger` 的同步条件列可在事件生成时过滤，可用于「只同步某个区域、某个状态的数据」。列级与值级：转换（transform）能力可在传输前改写列集与字段值，测试环境脱敏也可借助该机制实现。多主双向场景通过双向 `sym_node_group_link` 与冲突解决策略配置实现，具体策略项以官方 User Guide 为准。

### 6.4 适用边界与运维注意

适用判据：需要长期双向同步、节点数量多且网络不稳定（门店、边缘）、或托管数据库不允许配置日志级 CDC。不适用判据：一次性大表搬迁（触发器捕获在初始大批量阶段开销与复杂度都高于批式工具）、以及追求秒级以内延迟的高吞吐流式链路。运维上需要关注：`sym_data` 及批次表的增长与清理由内置任务负责，但需按官方建议配置保留期；触发器对写延迟的影响应在业务量高峰时段实测；社区版升级时注意配置表结构变更的兼容说明。

---

# 第三篇 专题进阶篇

## 第 7 章 部分表与条件筛选迁移专章

第二篇各方案已分别给出筛选用法，本章将跨方案的能力汇总为对照表，并补充生产向测试分发场景的完整做法。章节目的有二：为选型提供「筛选能力」维度的精确依据；给出筛选迁移特有的正确性问题清单（漏表、条件不一致、分片与条件冲突等）。

### 7.1 筛选能力对照总表

| 需求 | 原生工具（MySQL 系） | pg_dump / psql | Addax | pgloader | SeaTunnel | Debezium |
|---|---|---|---|---|---|---|
| 选择部分表 | `mydumper -T db.t1,db.t2`；`--regex` 正则 | `pg_dump -t t1 -t t2`；`-n` 按模式 | 作业按表配置 | `INCLUDING ONLY TABLES MATCHING` | `table-names` 列表；`table-pattern` 正则 | `table.include.list` |
| 排除部分表 | `-O /path/exclude.txt`（文件内每行一条 `db.table`） | 不支持反向（先全列表） | 不在作业中配置即可 | `EXCLUDING TABLES MATCHING` | 正则取反或改用列表 | `table.exclude.list` |
| 行级条件筛选 | `mydumper --where`；mysqldump `--where` | `\copy (SELECT ...)` 任意查询 | `where` 参数；`querySql` | 不支持 | `query`；CDC 快照有限 | `snapshot.select.statement.overrides`；信号快照附加条件 |
| 列裁剪 | 配置文件 `columns_on_select` | 不支持（整表导出） | `column` / `excludeColumn` | 按 cast 映射 | CDC 列投影；JDBC `query` | SMT 投影或下游过滤 |
| 分片并行与条件组合 | `-r` 行数切分 | `pg_dump -j` 仅整库并行 | `splitPk` 与 `where` 组合正确下推 | 内部并行 | 快照按主键切分 | 增量快照按主键分块 |

使用任何筛选组合前，建议先在源端手工执行等价的 `SELECT` 语句确认结果集规模与条件语义（尤其是时区、大小写与 `NULL` 处理差异），再写入迁移配置。

### 7.2 常见正确性问题

**条件在两端语义不一致。**同一表达式在 MySQL 与 PostgreSQL 中可能有不同行为，例如字符串比较的排序规则、隐式类型转换、日期字面量解析。标准做法是：筛选条件由源端方言编写并在源端执行，目标端校验时使用等价改写后的条件。

**全量筛选与增量无法对齐。**按条件（如 `created_at >= '2025-01-01'`）迁移部分数据后，若继续用 CDC 追增量，后续新增的行可能不满足原条件。此时有三种处理：把筛选条件定义为随时间推进的滑动窗口并周期性重跑全量；在 CDC 链路下游按同一条件做流式过滤（SeaTunnel Transform、Flink SQL `WHERE`、Debezium SMT）；或接受「筛选仅约束全量、增量全量入」并确保目标端无外泄路径。方案必须在迁移设计阶段明确，不能事后补救。

**分片与条件的交互。**批式工具的分片逻辑会把 `where` 条件下推到每个分片查询，通常无需人工干预；但使用自定义 `querySql` 后分片失效，大表必须改回 `where` 加 `splitPk` 组合。

**部分表迁移时的外键与依赖。**只迁移部分表时，目标端外键、视图与存储过程若引用了未迁移的表，将建不成立或运行报错。建议的目标端顺序为：先建全部表结构，再装数据，最后补外键与索引；引用缺失表的依赖对象在迁移清单中显式标记为「暂缓」。

### 7.3 生产向测试环境的完整分发流程

该场景的完整流程以 Addax 或 SeaTunnel 周期任务为核心，步骤如下。

1. **圈定范围**：与业务方确认表清单与行级条件（通常为最近数月数据），清单写入版本管理的配置文件。
2. **结构对齐**：测试库结构由发布流程统一管理，迁移任务只装数据（`schema_save_mode` 配置为报错或忽略，避免任务自动建表造成结构漂移）。
3. **脱敏规则**：列出敏感字段清单，明确每类字段的脱敏算法（确定性哈希保关联、截断、置空）。使用 Addax transformer 的 Groovy 脚本或 SeaTunnel 的 SQL Transform 实现。
4. **执行与校验**：任务前置清空目标表（`preSql` 执行 `TRUNCATE`），后置执行行数与抽样校验，校验脚本输出到任务日志。
5. **访问控制**：测试库对开发者开放写权限，与生产凭据完全隔离；脱敏后数据不得可逆还原。

该流程定时执行即为测试环境数据刷新机制；若测试需要持续与生产保持一致，则改用方案三的 CDC 链路并在下游加脱敏变换，同时严格评估合规要求（部分法规禁止生产个人数据离开生产环境，此时应以合成数据替代）。

---

## 第 8 章 异构类型映射与 Schema 演进

异构迁移中一半的故障来自类型与结构转换。本章给出三大库之间的常用类型映射基线、结构对象的转换策略，以及各工具的结构演进能力对照。

### 8.1 常用类型映射基线

以下映射为工程实践中的常用选择，覆盖绝大多数业务表；遇到特殊类型时以工具自动映射结果与官方映射文档为准。

**MySQL 到 PostgreSQL**：

| MySQL 类型 | 建议映射 | 说明 |
| --- | --- | --- |
| `TINYINT` | `SMALLINT` | 有符号即可覆盖 |
| `TINYINT(1)` | `BOOLEAN` | 布尔语义；Addax/pgloader 均可自动处理 |
| `SMALLINT` | `SMALLINT` | 直接对应 |
| `MEDIUMINT` | `INTEGER` | PostgreSQL 无中间档整型 |
| `INT` | `INTEGER` | 直接对应 |
| `BIGINT UNSIGNED` | `NUMERIC(20,0)` | PostgreSQL 无无符号整型，超出 `BIGINT` 上限时必须扩为数值类型 |
| `INT UNSIGNED` | `BIGINT` | 放宽位宽保住取值范围 |
| `DECIMAL(p,s)` | `NUMERIC(p,s)` | 直接对应 |
| `FLOAT` / `DOUBLE` | `REAL` / `DOUBLE PRECISION` | 直接对应 |
| `DATETIME` | `TIMESTAMP` | 若业务含跨时区语义建议 `TIMESTAMPTZ` |
| `TIMESTAMP` | `TIMESTAMPTZ` | MySQL 的 `TIMESTAMP` 本身带时区转换语义 |
| `DATE` / `TIME` | `DATE` / `TIME` | 直接对应 |
| `CHAR(n)` / `VARCHAR(n)` | `CHAR(n)` / `VARCHAR(n)` | 注意字符长度语义均为字符数 |
| `TEXT` 系 | `TEXT` | PostgreSQL 统一为 `TEXT` |
| `BLOB` 系 / `VARBINARY` | `BYTEA` | 二进制统一为 `BYTEA` |
| `ENUM` | `VARCHAR` 加 CHECK 约束，或 `ENUM` 类型 | 建议前者，便于演进 |
| `JSON` | `JSONB` | `JSONB` 支持索引 |
| `AUTO_INCREMENT` | `BIGSERIAL` / `IDENTITY` 列 | 序列值迁移后需校准 |
| `GEOMETRY` | `GEOMETRY`（PostGIS 扩展） | 需先安装扩展 |

**SQL Server 到 PostgreSQL**：

| SQL Server 类型 | 建议映射 | 说明 |
| --- | --- | --- |
| `BIT` | `BOOLEAN` | |
| `TINYINT` | `SMALLINT` | SQL Server 无 `SMALLINT` 以下的整型 |
| `IDENTITY(1,1)` | `BIGSERIAL` / `IDENTITY` | 序列值校准 |
| `DATETIME2` | `TIMESTAMP` | 精度按需指定 |
| `DATETIMEOFFSET` | `TIMESTAMPTZ` | |
| `UNIQUEIDENTIFIER` | `UUID` | |
| `NCHAR` / `NVARCHAR` | `CHAR` / `VARCHAR` | PostgreSQL 文本无宽窄之分 |
| `MONEY` / `SMALLMONEY` | `NUMERIC` | |
| `VARBINARY(MAX)` | `BYTEA` | |

**MySQL 到 SQL Server**：`AUTO_INCREMENT` 对应 `IDENTITY(1,1)`；`TINYINT UNSIGNED` 放宽为 `SMALLINT`；`DATETIME` 对应 `DATETIME2`；`JSON` 以 `NVARCHAR(MAX)` 承载；`ENUM` 改为字典表加外键或 CHECK 约束。

## 8.2 结构对象转换策略

**自增与序列。**MySQL 的 `AUTO_INCREMENT` 与 PostgreSQL 的序列是两套机制。迁移数据后必须把目标序列的当前值校准到已迁数据的最大主键值，否则新插入将报主键冲突；pgloader 的 `reset sequences` 选项可自动完成，手工迁移时用 `setval` 显式校准。SQL Server 的 `IDENTITY` 同理，插入后需执行 `DBCC CHECKIDENT` 类操作或按官方方法校准当前值。

**外键、索引与约束。**推荐顺序为「先建表、后装数、再建索引与外键」，理由是装载阶段跳过约束校验与索引维护可显著提速，且行级筛选迁移场景下外键可能因引用表未迁移而无法建立。

**程序性对象。**视图语法差异较小（重写量大而概念一致），存储过程、函数与触发器是方言重写工作量最大的部分，任何工具都无法可靠自动转换。工程上应将它们从「数据迁移」剥离，纳入应用发布流程单独迁移与回归测试。

**字符集与排序规则。**MySQL 侧确认库表字符集为 `utf8mb4`，PostgreSQL 侧建议目标库直接使用 UTF-8 编码；排序规则（collation）差异会影响唯一索引对大小写与重音字符的判定，迁移前应明确目标排序策略。

### 8.3 各工具的 Schema 演进能力

实时同步期间源端执行 DDL 是高频故障来源。下表对照各工具的结构演进机制（能力细节以各官方文档为准）。

| 工具 | 机制 | 说明 |
| --- | --- | --- |
| SeaTunnel | `schema-changes.enabled = true` | MySQL 源端支持加列、删列、改列名、改列类型同步到目标端 |
| Debezium（Kafka Connect） | schema history 主题 | 连接器记录 DDL 历史保证事件结构正确解析；目标端结构变更由 Sink 连接器（如 `schema.evolution = basic`）或人工执行 |
| Debezium Server | 同上，JDBC Sink 支持 `schema.evolution` | 基本结构演进 |
| Flink CDC | Schema Evolution 框架 | 多表整库同步场景下随源端结构变化调整目标表 |
| Canal | 无内置 | DDL 事件可见，需消费端自行处理 |
| SymmetricDS | 触发器重建机制 | 表结构变更后需重建触发器并处理兼容 |
| 批式工具（Addax 等） | 不适用 | 结构由外部流程管理，任务仅搬数据 |

生产建议：迁移窗口内冻结 DDL；窗口外的常规 DDL 变更走「先目标端、后源端」或「先源端、后目标端」的双端发布流程（顺序由变更类型决定，加列通常先目标端，删列先源端），并把 DDL 纳入迁移运行手册。

---

## 第 9 章 性能调优与大规模迁移

大规模迁移的调优目标是三个可控变量：源端读取吞吐、网络传输吞吐、目标端装载吞吐。三者中的最短板决定总吞吐。本章按工具给出调节手段与典型瓶颈定位方法。

### 9.1 批式链路调优

**源端读取。**Addax 的 `splitPk` 或 `autoPk` 开启并行抽取，分片数与 `channel` 数共同决定并发度；分片键选择数据分布均匀的整型主键可避免数据倾斜。`fetchSize` 控制单次网络往返拉取行数，适当调大可减少往返次数，但过大将抬高内存占用。mydumper 以 `-r` 控制行数切分、`-t` 控制线程数；pg_dump 以 `-j` 控制目录格式导出的并行度。

**网络与限速。**跨机房传输时先实测链路带宽与延迟，再据此设置限速项（Addax `speed.byte`、SeaTunnel `read_limit.bytes_per_second`），保证生产库只让渡固定份额的资源。压缩传输（若工具支持压缩或链路支持压缩协议）对文本类数据的传输提升明显。

**目标端装载。**经验顺序为：`COPY` 类协议最快（SeaTunnel 对 PostgreSQL 开启 `use_copy_statement`），批量 `INSERT` 次之，逐条 `INSERT` 最慢。装载前延后建索引与外键，装载后补建（`create_index` 类参数在自动建表场景下应设为延后）。批量提交大小（`batchSize`、`batch_size`）在 1000 至 10000 行区间试验取优。PostgreSQL 目标端装载完成后执行 `ANALYZE`；MySQL 目标端大批量插入期间可临时调大事务批量与缓冲参数（以官方手册为准，不做通用推荐）。

**典型瓶颈定位。**源端 CPU 或 IO 打满，先降低抽取并发；网络带宽打满，开启压缩或降低并发；目标端为瓶颈时，检查写入是否逐条提交、索引是否未延后、锁等待是否频繁。调优顺序遵循「先定位最短板，再单点调节，每次只改一个变量并复测」。

### 9.2 CDC 链路调优

**快照阶段。**Debezium 的 `snapshot.max.threads` 开启并行快照（SQL Server 连接器已支持按主键区间的分块并行快照），`incremental.snapshot.chunk.size` 控制增量快照分块行数；SeaTunnel 的 `snapshot.split.size` 与 `parallelism` 同理。快照阶段的写入压力集中，务必在业务低峰执行并限速。

**流式阶段。**下游批量应用是关键：Debezium JDBC Sink 的批处理与 reduction buffer 可减少逐条写入；SeaTunnel 的 `batch_size` 与目标端 upsert 索引效率决定应用速度。Kafka 侧关注分区数与消费并行度匹配（同表事件按主键保序，跨表可并行）。

**源端保护。**所有 CDC 采集对源端的增量开销小于批量扫描，但快照阶段等同一次全表读取。生产库执行全量快照的标准防护为：从只读副本读取快照（工具支持时）、低峰窗口执行、限速执行。MySQL 侧还需关注 `interactive_timeout` 与 `wait_timeout`（大库快照耗时长，连接可能被服务端提前关闭，适当调大）。

### 9.3 特殊对象处理

**大对象（BLOB 与超长文本）。**大对象同时拖慢三类吞吐。处理原则：能不迁就不迁（归档到对象存储、目标端存引用）；必须迁移时用专用大对象模式（Debezium 对超大事件提供大消息处理机制，SeaTunnel 与 Addax 对二进制列有对应类型映射，注意 Addax 文本输出对二进制列采用 base64 编码的约定）。逐行估计平均行宽，过大时降低分片行数。

**无主键表。**CDC 工具需要主键或唯一键定位行，无主键表的更新与删除无法可靠应用。处理选项：迁移前补建主键（首选）；Debezium 可通过 `message.key.columns` 类机制人为指定键（效果有限）；SeaTunnel 对个别表通过 `table-names-config` 显式声明主键。无键表在 CDC 链路中建议只做追加式同步。

**大表与热点。**数亿行级表单独成任务、单独调度与校验；自增键严重空洞或分布倾斜时，改用采样切分（SeaTunnel 与 Debezium 均内置采样策略）；跨分片一致性无要求时，可按主键区间人为拆分为多个作业并行执行。

---

# 第四篇 工程精通篇

## 第 10 章 生产级割接与回滚

本章把前九章的工具能力组织为一次可执行的迁移工程。核心观点：迁移的成功不取决于工具先进程度，而取决于割接窗口与回滚路径的设计质量。全章以「生产 MySQL 迁往新环境 PostgreSQL、要求业务停写不超过十分钟」为基准场景展开，其余场景按同一框架裁剪。

### 10.1 迁移总体状态机

把迁移建模为带门禁的状态序列，每个状态都有进入条件、观测手段与退出动作，未通过门禁不得进入下一状态。




### 10.2 割接窗口内必须完成的动作

停写围栏开启后到切流完成之间，按顺序执行以下动作，建议编为带勾选框的脚本化清单。

1. **停写围栏**：应用侧将源库置为只读（连接池切换只读用户或开启只读标记），业务返回友好提示。必须杜绝「停止大部分写入」的模糊状态。
2. **等待增量追平**：观察 CDC 延迟指标归零，确认目标端已消费到最后一个变更。
3. **同步序列与外部状态**：PostgreSQL 目标端对全部序列执行 `setval` 校准；同时核对缓存、搜索索引、对象存储引用等外部状态。
4. **最终校验**：对关键表执行行数与聚合快查，与预演结果对照。
5. **切流**：应用连接串指向目标库，先切读、验证后切写；或按灰度比例逐批放量。
6. **记录窗口时长**：从停写到切写完成的总耗时，作为演练目标值与复盘基线。

### 10.3 双写与灰度

对一致性要求极高的核心系统，可在 CDC 追平后增加双写阶段：应用同时写源库与目标库，以源库为准、目标库异步跟随，比对双写结果作为最终校验。双写的工程代价是应用改造与补偿逻辑，是否采用取决于业务对一致性与复杂度的权衡。灰度切流（按用户、按机房、按流量比例）是降低爆炸半径的通用手段，每次放量后执行一轮业务指标核对再继续。

### 10.4 回滚设计

回滚路径必须在割接前建成并演练，包含三个部分。其一，**流量回退**：连接串或路由层切回源库，源库在观察期内保持只读但结构完整。其二，**数据回灌**：观察期内目标端产生的新写入需要带回源库，常规做法是在割接时同步建立反向 CDC 链路（SeaTunnel 与 Debezium 均可反向配置），回退时开启；未建反向链路时以手工补偿为兜底，必须明确补偿范围。其三，**决策时限**：明确「观察多长时间、超过即视为跨过不可逆点、此后只能前滚修复」。回滚演练应作为割接演练的一部分实际执行。

### 10.5 运行手册与演练

把以上内容固化为迁移运行手册（runbook），至少包含：对象清单与责任矩阵、每个状态的进入与退出条件、全部命令与预期输出、故障分支的处理动作、回滚剧本、通讯与升级路径。正式割接前至少完成一次全流程演练（在测试环境按生产同构环境执行），演练通过是进入正式割接的前置门禁。

---

# 第 11 章 数据校验、监控与容错

迁移价值的最终证明是数据一致性。本章给出校验的分层方法、监控指标体系与故障恢复要点。

## 11.1 分层校验方法

| 层次 | 方法 | 成本 | 捕获的问题 |
| --- | --- | --- | --- |
| 结构校验 | 比较两端表清单、列清单与索引清单 | 低 | 漏表、漏列、类型映射偏差 |
| 行数校验 | 两端 `COUNT(*)`，筛选场景两端同条件 | 低 | 漏数、多数、条件语义不一致 |
| 聚合校验 | 对数值列 `SUM`、日期列 `MIN` / `MAX`、状态列分组计数 | 低 | 静默截断、精度损失、布尔语义翻转 |
| 抽样校验 | 按主键分层抽样，逐列比对明细 | 中 | 行内字段错位、编码问题 |
| 全量比对 | 两端按主键排序做逐行校验和 | 高 | 抽样覆盖不到的零星差异 |

实现要点如下。聚合校验的 SQL 应在两端各自执行，避免依赖跨库函数；跨库校验和因浮点求和顺序与字符串排序差异可能不可比，此时改用逐列聚合。全量比对建议按主键分桶计算（对每桶行集做聚合指纹），差异桶再细查，避免一次性全表排序。校验脚本输出统一为机器可读结果（如 CSV），纳入任务产物归档。割接前的最终校验应覆盖「结构、行数、聚合、抽样」四个层次；CDC 长期同步的巡检可降低频次但不应取消。

## 11.2 监控指标体系

| 指标 | 来源 | 告警建议 |
| --- | --- | --- |
| 端到端延迟 | Debezium `MilliSecondsBehindSource` 等官方指标；SeaTunnel、Flink CDC 检查点与积压指标 | 超过业务容忍阈值告警 |
| 位点健康 | MySQL binlog 保留期与当前位点距离；PostgreSQL `pg_replication_slots` 槽滞后与 WAL 目录增长；SQL Server CDC 表保留窗口 | 接近日志保留期高优告警 |
| 任务状态 | 调度系统任务成败、Addax 错误计数、SeaTunnel 检查点失败次数 | 失败即告警并自动重试一次 |
| 目标端写入 | 目标库死锁、锁等待、连接池饱和 | 按目标库常规 DBA 阈值 |
| 校验结果 | 周期校验任务的差异计数 | 任何非零差异告警 |

告警应绑定处理动作：延迟升高先看源端与网络，再看目标端应用速度；位点逼近保留期立即扩保留或恢复消费。

## 11.3 故障恢复要点

**采集端故障。**SeaTunnel 与 Flink CDC 依赖检查点恢复，检查点间隔即最大重复窗口；Debezium 位点存于 Kafka，自然高可用；Debezium Server 位点存本地文件，需保证目录持久化。恢复后的第一动作是确认位点未越过日志保留期，越界则须重做全量。

**目标端故障。**写入失败按批重试；注意非幂等写入在重试下会放大重复，因此 5.6 节的 upsert 要求在此处生效。目标端长时间不可用时，先暂停采集（避免位点堆积过快）再修复，恢复后观察延迟回落的斜率。

**源端主从切换。**源库发生主备切换后，MySQL 的 binlog 位点在新主上可能不可续接（依赖 GTID 时可自动衔接，SeaTunnel 与 Debezium 均支持 GTID），PostgreSQL 复制槽默认不随主从切换漂移，需按官方文档处理槽的迁移或重建。该场景应纳入运行手册的故障分支。

---

# 第 12 章 综合实战与自动化

本章以端到端案例串联全书：将生产环境 MySQL（`shop` 库，约 800 GB，包含大表 `orders`）迁移到新环境的 PostgreSQL，要求业务停写不超过十分钟。随后给出把该流程固化为自动化流水线的骨架。

## 12.1 案例方案设计

**选型**：目标为关系库且要求不停服，主方案选 SeaTunnel（MySQL-CDC 到 JDBC Sink 一体化全量加增量）；备选为 Debezium Server JDBC 直写。结构迁移采用「pg_dump 式结构导出加人工类型映射」不行（MySQL 无此产物），实际采用人工映射脚本：从 MySQL information_schema 读取列定义，按第 8 章映射基线生成 PostgreSQL DDL，评审后执行。

**前置条件清单**：

```
[ ] MySQL：binlog_format=ROW，binlog_row_image=FULL，日志保留 7 天
[ ] MySQL：创建 cdc_user 并授予 SELECT，RELOAD，SHOW DATABASES，REPLICATION SLAVE，REPLICATION CLIENT
[ ] PostgreSQL：wal_level=logical（SeaTunnel JDBC Sink 不强制，但便于后续反向链路）
[ ] PostgreSQL：目标库与账号创建；max_prepared_transactions 大于 0（若启用 exactly-once）
[ ] 网络：SeaTunnel 节点到两端库端口连通；带宽实测
[ ] 目标端：建表脚本执行并评审；外键与二级索引延后
```

**执行序列**：

1. 启动 SeaTunnel 流式任务，`startup.mode = "initial"` 自动完成一致性快照并进入增量。`orders` 表依赖按主键分块并行快照，预计全量阶段约数小时，全程业务无感知。
2. 观察延迟指标，等待端到端延迟稳定低于 2 秒并持续 30 分钟。
3. 执行分层校验：结构、行数、聚合、抽样四层脚本全部通过。
4. 按第 10 章剧本完成演练与正式割接：停写围栏、延迟归零、序列 `setval` 校准、补建索引与外键、切流。
5. 观察期 72 小时，反向回滚链路在位；期满后执行收尾：源库转只读归档、删除复制槽与临时账号、归档迁移产物。

## 12.2 自动化流水线骨架

把迁移工程固化为可重复流水线的目录结构建议如下。

```
migration/
  inventory/          # 对象清单与范围配置（表清单、筛选条件、脱敏规则）
  schema/             # 目标端 DDL 与映射脚本
  jobs/               # Addax JSON / SeaTunnel conf / 连接器配置
  scripts/            # 导出、导入、序列校准、校验脚本
  runbook.md          # 运行手册（状态、门禁、命令、回滚）
  checks/             # 校验 SQL 与预期结果
```

流水线的最小自动化骨架（伪代码，按所选用调度系统实现）：

```
阶段 1  preflight    执行前置条件检查脚本，全部通过才继续
阶段 2  schema       幂等执行目标端建表（IF NOT EXISTS 或先检后建）
阶段 3  bulk         启动全量迁移任务（或等待 CDC 任务快照阶段完成）
阶段 4  verify_full  行数加聚合加抽样校验，输出差异报告
阶段 5  sync_watch   持续监控 CDC 延迟，达标后输出放行标记
阶段 6  cutover      人工门禁确认后执行割接脚本（含序列校准）
阶段 7  post_verify  割接后校验与观察期巡检
```

工程化要点：所有作业配置进版本管理，凭据走密钥管理服务；每个阶段输出机器可读产物（校验报告、延迟快照）便于审计；「人工门禁」在自动化中表现为显式的审批等待点，不允许自动化直接执行割接。

## 12.3 常见故障与处置速查

| 现象 | 高概率原因 | 处置 |
| --- | --- | --- |
| CDC 启动报日志不可读 | binlog 已被清理或格式非 ROW | 扩保留期重做全量；核对 ROW 与 FULL |
| PostgreSQL 端 WAL 目录持续增长 | 复制槽长期无消费 | 恢复消费或按需释放槽 |
| 全量阶段连接中断 | 服务端超时参数过小 | 调大 `wait_timeout` 等会话参数，任务断点续传 |
| 目标端序列值冲突 | 序列未校准 | `setval` 到最大主键值 |
| 行数校验差异为固定小数目 | 快照与增量衔接期间的边界行 | 以延迟归零后的重校验为准，必要时重同步该表 |
| 中文乱码 | 字符集与连接编码不一致 | 双端统一 UTF-8，核对连接参数 |
| 大表任务超时 | 分片过大或单通道 | 减小分片、增加并发、延长超时 |

# 附录

## 附录 A 工具速查表

| 工具 | 官方地址 | 最新稳定版（核实日期 2026-10-25） | 许可证 | 一句话定位 |
|---|---|---|---|---|
| Addax | github.com/wgzhao/Addax | 6.2.0 | Apache-2.0 | DataX 活跃后继，批式异构 ETL |
| Apache SeaTunnel | seatunnel.apache.org | 2.3.13（3.0.0 已发布） | Apache-2.0 | 一体化全量加增量数据集成引擎 |
| Debezium | debezium.io | 3.7.0.Final | Apache-2.0 | 基于 Kafka Connect 的 CDC 平台 |
| Debezium Server | debezium.io | 同 Debezium | Apache-2.0 | 免 Kafka 独立 CDC 运行时，JDBC 直写 |
| Apache Flink CDC | flink.apache.org | 3.7.0 | Apache-2.0 | Flink 生态 CDC 集成与整库流水线 |
| pgloader | github.com/dimitri/pgloader | 3.6.x（v4 预发布） | PostgreSQL Licence | 一步迁入 PostgreSQL |
| Canal | github.com/alibaba/canal | 1.1.8 | Apache-2.0 | MySQL binlog 增量订阅与投递 |
| mydumper / myloader | github.com/mydumper/mydumper | 1.0.x | GPL-3.0 | MySQL 多线程逻辑导出导入 |
| SymmetricDS 社区版 | symmetricds.org | 3.16.x | AGPL-3.0 | 触发器捕获的多节点双向同步 |

## 附录 B 术语表

| 术语 | 释义 |
|---|---|
| binlog | MySQL 二进制日志，记录行级或语句级变更，是 MySQL 系 CDC 的数据来源 |
| WAL | PostgreSQL 预写日志；逻辑解码（logical decoding）将其解码为逻辑变更流 |
| LSN | Log Sequence Number，PostgreSQL 与 SQL Server 中标识日志位置的序列号 |
| GTID | 全局事务标识符，MySQL 中跨实例唯一标识事务，便于位点管理与主从切换衔接 |
| 复制槽（replication slot） | PostgreSQL 中保存消费进度的服务端对象，防止 WAL 过早回收 |
| 发布 / 订阅 | PostgreSQL 逻辑复制的生产消费模型 |
| publication / subscription | 同上英文原词 |
| 快照（snapshot） | CDC 工具在启动时对源库做的一致性全量读取 |
| 增量快照（incremental snapshot） | 在流式消费不停顿的前提下，按主键分块补充执行的全量读取 |
| 信号表（signaling table） | Debezium 用于接收快照等控制指令的数据库表 |
| SMT | Single Message Transform，Kafka Connect 中对单条消息执行的转换 |
| upsert | 按主键存在则更新、不存在则插入的写入语义 |
| 检查点（checkpoint） | 流式任务的周期性状态持久化，用于故障后恢复 |
| 割接（cutover） | 业务流量从源库切换到目标库的动作 |
| 反向同步 | 割接后从新库向旧库回传变更的链路，用于回滚保障 |
| 脱敏（masking） | 对敏感字段做不可逆变换后交付非生产环境 |

# 附录 C 迁移检查清单

### 迁移前

```
[ ] 明确迁移类型：同构/异构，一次性/持续，停机预算
[ ] 对象清单：库、表、行数、最大表宽、特殊对象（大对象/无主键/序列/存储过程）
[ ] 类型映射评审通过，DDL 评审通过
[ ] 源端前置条件：日志开启、权限账号、保留期、只读副本可用性
[ ] 目标端：容量规划、建表脚本、索引延后策略、参数（如 max_prepared_transactions）
[ ] 工具选型确定，版本核实，试跑通过
[ ] 校验脚本就绪（结构/行数/聚合/抽样四层）
[ ] 运行手册完成，回滚路径明确，演练排期
```

### 迁移中

```
[ ] 全量任务限速与进度监控正常
[ ] 增量延迟稳定达标，位点健康
[ ] DDL 冻结生效，双端变更流程在位
[ ] 周期校验无差异
```

### 割接

```
[ ] 演练通过并复盘
[ ] 停写围栏脚本就绪，应用侧确认
[ ] 序列校准命令就绪
[ ] 最终四层校验通过
[ ] 回滚链路（反向同步或手工补偿方案）确认在位
[ ] 通讯与升级路径确认
```

### 割接后

```
[ ] 观察期业务指标正常，延迟与错误率无异常
[ ] 反向链路运行正常
[ ] 观察期满后收尾：源库转只读、删槽删账号、归档迁移产物
[ ] 复盘文档归档，更新运行手册
```

# 附录 D 版本核实来源说明

本教程关键事实的核实来源如下（核实日期均为 2026-10-25）：Addax 版本与 DataX 冻结状态取自 Addax 官方仓库 Releases 页面与 README；SeaTunnel 版本与连接器配置取自 Apache SeaTunnel 官方下载页与 2.3.13 版文档（MySQL CDC 源端、JDBC Sink）；Flink CDC 版本与 Pipeline 连接器支持矩阵取自 Apache Flink 官方 3.7.0 发布公告、官方下载页与连接器总览文档；Debezium 版本、SQL Server 前置条件与快照机制取自 Debezium 官方 Releases Overview 与 3.7 版 SQL Server 连接器文档，Debezium Server JDBC Sink 取自 Debezium 官方文档与官方博客；pgloader 能力与 v4 状态取自 pgloader 官方仓库 README 与官方站点；Canal 版本与特性取自其 GitHub Releases 页面与官方 Wiki；mydumper 选项取自官方文档站；SymmetricDS 定位与许可证取自官方网站、FAQ 与 GitHub 仓库说明；PostgreSQL 逻辑复制与逻辑解码机制取自 PostgreSQL 官方文档体系；SQL Server CDC 功能的版本与 edition 要求取自 Debezium 官方文档所引 Microsoft 发布说明。凡本教程未能核实的细节均已标注「以官方文档为准」，使用前请复查对应官方页面。